
Direct Logic Arithmetization, Corrected

Sound finite-field compilation, finite higher-order polynomial semantics, and proof objects beyond polynomials

THE CLAIM

Direct first-order arithmetization promises an unusually short route from logic to polynomial constraints.

THE RESULT

A formal construction suite for sound direct logic: corrected FOL, finite higher-order semantics, live DREGG descriptors, executable conformance, and encrypted evaluation.

EVIDENCE STANDARD

This report distinguishes proved mathematics, checked executable models, theorem interfaces, implementation evidence, and future work. Performance claims are not inferred from algebra alone.

Contents

1	What was built, what failed, what got faster	3
1.1	The construction stack	3
1.2	Performance evidence we can actually claim	4
1.3	The public-audit result	5
2	Audit of the direct FOL arithmetization claim	6
2.1	What Gabbay actually proves	6
2.2	Why the naive finite-field port fails	6
2.3	The BitLogic connective reversal	7
2.4	Evidence boundary and constructive next step	8
3	Corrected constructions: from matrices to searched proof plans	10
3.1	Construction map	10
3.2	Exact matrix semantics and its compiler	10
3.3	The field boundary is a certificate, not a cast	11
3.4	Two routes into the live DREGG relation	12
3.5	Checked optimization with exact savings	13
3.6	Exact presentations and finite search	14
4	Higher-order logic and the categorical boundary	14
4.1	Finite extensional HOL: complete, exact, and dense	15
4.2	Locally finite categories: the exact indexed classification	16
4.3	The intensional route: a shared-net CCC	16
4.4	From typed reductions to checked interaction receipts	17
4.5	Arbitrary small CCCs: Yoneda is the semantic generalization	17
4.6	The presentation category correction	18
5	Proof objects beyond polynomials:	
	realizability, descent, and transcript soundness	20
5.1	Realizability is a semantics of evidence, not a field trick	20
5.2	Presentations form fibers, not one global CCC	22
5.3	Exact descent: compatible local evidence glues uniquely	22
5.4	Hidden-query soundness: price the obstruction that was missed	23
5.5	Full transcript composition, including BabyBear bias	23
5.6	Local interaction traces are proof objects too	25
6	From finite logic to a live proof and encrypted plan	26
6.1	General finite FOL reaches the live descriptor IR	26
6.2	Certificates optimize the relation without changing it	27
6.3	One opening, proof and FHE	28
6.4	Executable backends	29
6.5	Conformance before performance	31
6.6	What would test the claimed performance numbers	31
6.7	Constructive public comparison	32
7	Conclusions: from a broken shortcut to a certified compiler family	33
7.1	The corrected answer to the original claim	33
7.2	The novel contribution that emerged	33
7.3	Higher order and the CCC question	34
7.4	Proof objects beyond polynomials	34
7.5	Executable assurance now present	35
7.6	What may be claimed now	35
7.7	Constructive offer	36
8	Primary sources and inspected artifacts	36

SECTION 1

What was built, what failed, what got faster

Direct logic arithmetization is salvageable, generalizable, and useful. The reviewed public BitLogic presentation is not yet a sound instance of it. This report does more than separate those statements: it supplies the missing constructions.

RESULT We reconstructed Gabbay’s matrix-language compiler, proved its characteristic- zero semantics, added two exact finite-field repairs, and constructed a fixed-shape private-witness matrix relation in DREGG’s live DescriptorIR2. We then built a general finite-signature FOL compiler, a checker-gated optimizer, a same-opening proof/FHE certificate, an intensional cartesian closed compiler with explicit sharing, a presheaf embedding for arbitrary small CCCs, and a bias-aware robust transcript theorem. The public BitLogic equations still fail the permanent counterexample suite. The positive stack does not.

1.1 The construction stack

Layer	Verified construction	Exact boundary
Gabbay front end	Integer matrices, one-based Lagrange row interpolation, terms, bounded quantifiers, source semantics, rational residual compiler, projection certificate, and Boolean repair.	<code>GabbayMatrixCompiler</code> and <code>GabbayMatrixBridge</code> ; positive one-index matrix fragment, not arbitrary recursion.
Live matrix relation	A three-entry Skolem successor table has a constructive accepting trace in actual <code>EffectVmDescriptor2</code> . Coefficient columns are checked, while the residual and acceptance gates read the private table cells directly.	<code>GabbayDescriptorIR2PublicBoundary</code> proves <code>piCount = 0</code> , with no hash or range sites: this is an existential witness relation, not an attestation of an external table or evidence of interpolation performance.
Finite FOL	Closed de Bruijn syntax with equality, many relations of independent arities, total public function tables, every connective, and exhaustive finite quantification.	<code>FiniteSignatureFOLDescriptorIR2</code> ; exact canonical layout, live relation, certificate, and emitted bytes.
Certified optimization	Local rules reconstruct both endpoints; a fail-closed checker proves semantic equivalence and coordinatewise cost nonincrease; the recursive optimizer emits its own certificate.	<code>DirectLogicOptimizerCertificate</code> ; an independent reference tool also performs checked Pareto search across residual and Boolean presentations.

Layer (continued)	Verified construction	Exact boundary
Proof and FHE	One raw opening feeds the proof relation and a BFV plan through an explicit conversion witness. A mixed plan carries exact resource counts and an acceptance-equivalence theorem.	<code>CertifiedHybridProofFhe</code> ; algebraic and scheduling theorem, not a Rust-to-Lean refinement or a parameter-security theorem.
Higher order	Typed STLC compiles to shared nets with local beta, eta, and product evidence. The semantic quotient is a genuine CCC and the compiler is a strong-CCC functor.	<code>IntensionalCCCTrace</code> , <code>IntensionalCCCInteractionBridge</code> , and <code>IntensionalCCCCategory</code> ; exact for the formalized language.
General CCC boundary	Every small CCC embeds fully faithfully by Yoneda into presheaves, preserving terminal objects, products, and representable exponentials. Fixed-meaning evidence fibres are CCCs; the global category of meaning-changing presentations provably is not.	<code>SmallCCCYonedaPresheaf</code> and <code>CertifiedPresentationFibredCCC</code> ; no finiteness or finite-field succinctness is inferred.
Sampled proof objects	Commit-before-query descent with point binding gives an exact finite union bound. Modular query bias yields the sharp miss term $((1 - \delta) + \frac{\delta}{N})^k$ when $N \bmod m = 1$.	<code>RobustDescentTranscriptGame</code> ; OOD and low-degree errors remain explicit protocol sockets.

This changes the technical center of the report. FOL is a source language, not the primitive substance of proof. A compiler may preserve one judgment while changing its presentation locally: natural residual, Boolean graph, shared net, live AIR, or encrypted plan. The reusable theorem is the certified change of presentation, with semantics and resources preserved at every edge.

1.2 Performance evidence we can actually claim

The resulting gains are real but narrower than the advertised 10–100x claim. They are theorem-checked structural counts on named specimens, plus separately recorded executable measurements.

Construction	Before	After	Verified result
Symbolic BoolGraph optimizer	30 equations 26 multiplications 17 witnesses	13 equations 13 multiplications 8 witnesses	57%, 50%, and 53% fewer in the materialized BoolGraph ledger.
Formal hybrid BFV plan	9 multiplications depth 4	6 multiplications depth 2	33% fewer multiplications and half the depth, before one unpriced boundary zero decision.
Intensional sharing	16 work nodes 19 total nodes	8 work nodes 13 total nodes	50% less duplicated work and 32% fewer nodes on the beta-sharing specimen.

The optimizer counts above do not describe the current live `DescriptorIR2` encoding, which keeps a nested `WindowExpr` and materializes no intermediate BoolGraph columns. They therefore establish a certified optimization opportunity, not a current live-prover speedup. The hybrid `convertResidual` is likewise a semantic boundary: its encrypted residual zero-test is not implemented or priced as BFV arithmetic in this result. The real BFV path separately executed a Boolean workload in 22.950435 ms and an eight-equality residual workload in 48.197263 ms in one recorded warm release run. Those are different computations and SIMD occupancies, so they define no speedup ratio. The residual plan’s 8 multiplications and depth 1 compare symbolically with 15 multiplications and depth 4 for the matching all-Boolean eight-equality formula, but exclude the residual’s final private zero decision.

The current live finite-logic proof path now also has a reproducible absolute capture across seven small one-row workloads. On a contended Apple M2 Max, proof medians ranged from 2.22-26.83 ms, verification medians from 0.28-0.80 ms, and proofs from 9.3-13.1 KB. The medians are nonmonotonic and define no speedup, scaling law, Modulus comparison, or finality result.

The executable conformance corpus evaluated 10,061 cases. The corrected `D-NOWRAP` lane passed all 6,737 admitted cases and refused 3,324 unsupported ones; `D-BOOLGRAPH` and `C-AIR` passed all 10,061. `M-SPEC` produced 3,936 mismatches and the naive field projection produced one cancellation mismatch. Unsound lanes are retained as falsification artifacts and excluded from every performance ratio.

FORMAL ARTIFACT - executable boundary The Rust finite-logic front end compiles typed finite inputs, predicates, and quantifiers into the live descriptor grammar; its focused suite contains an honest prove-and-verify round trip and refusal tests. The independent Standard ML checker passed 15/15 version-1 fixtures, while HOL4 proves accepted bytes semantically sound and records zero theory axioms; proof-producing CakeML translation certificates exist for that checker. The reference compiler's checked hybrid search is independently executable. These are substantial implementation artifacts, but no native CakeML checker binary, optimizer-to-live-descriptor lowering theorem, or Rust-to-Lean refinement is claimed.

1.3 The public-audit result

The audited public claim does not survive technical review.

- Gabbay's construction is sound in its stated nonnegative characteristic-zero semantics. Naively interpreting its zero test in a prime field is unsound because nonzero residues can cancel or wrap to zero.
- BitLogic reverses the published zero-true connective and quantifier rules. Worse, its swap relation multiplies independent obligations. Under zero-true semantics multiplication expresses disjunction, so satisfying one factor can accept while the others fail. The concrete assignment $A = B = 10$, $k = q = 1$, $A' = 9$, $B' = 999$ is accepted by the displayed product although it violates two of the three intended obligations.
- The reviewed public repository supplies a PDF, not a runnable compiler, prover, verifier, parameter set, raw benchmark corpus, or reproducible path to the advertised 10–100x cost reduction and 1–3 second finality.

BOUNDARY The relation-level counterexample is decisive for the published equations; it is not a claim about unseen private code or a demonstrated production exploit. Conversely, unseen private code cannot substantiate a public performance or soundness claim. The strongest comparison currently available is therefore public evidence against public evidence: DREGG supplies the stronger formal semantics, executable falsification, construction breadth, and reproducibility; no like-for-like Modulus runtime comparison is possible from the released material.

EVIDENCE LEGEND **Kernel-checked** means a Lean or HOL4 theorem behind the stated axiom-hygiene gate. **Executable** means a concrete compiler, checker, counterexample, witness, or evaluator. **Measured** means a recorded implementation run with its workload and exclusions. A **theorem socket** still needs a protocol or backend instantiation. A **marketing claim** lacks a public reproducibility path.

SECTION 2

Audit of the direct FOL arithmetization claim

This section separates three statements that are easy to conflate: Gabbay’s characteristic-zero semantics, a finite-field implementation of that semantics, and the equations printed in the public ModulusZK/BitLogic whitepaper. The algebraic kernel of the first is sound in its stated model. A direct finite-field port is not sound without an additional invariant or gadget. More strongly, the published BitLogic encoding is not a finite-field implementation of Gabbay’s theorem: it changes the semantic domain and reverses every connective and finite quantifier fold used by that theorem. These are constructive specification findings: they neither allege bad intent nor establish a vulnerability in a deployed system.

The primary Gabbay source is the official [ePrint 2024/954 record](#), last revised 12 February 2026 and published in the *Journal of Applied Logics* 12(6), 1481–1548 (2025). Numbered references below follow the current ePrint text; theorem numbers, rather than unstable viewer page counts, are the primary locators. The BitLogic source is the 14-page PDF in the official public repository. Its pinned tree at commit

`26f125b37f0442d92991e3066095b1dfef1b0ce4` (21 November 2025) contains exactly that one PDF blob.

2.1 What Gabbay actually proves

Gabbay interprets predicates as polynomials in $\mathbb{Q}[X]$ whose values on $\mathbb{Q}$ are nonnegative. Zero denotes truth. Figure 2, defining the semantics in Definition 2.2.8, assigns addition to conjunction and bounded universal quantification, and multiplication to disjunction and bounded existential quantification:

$$\begin{aligned} R(\varphi \wedge \psi) &= R(\varphi) + R(\psi), & R(\varphi \vee \psi) &= R(\varphi)R(\psi), \\ R(\forall x \in H. \varphi(x)) &= \sum_{x \in H} R(\varphi(x)), & R(\exists x \in H. \varphi(x)) &= \prod_{x \in H} R(\varphi(x)). \end{aligned}$$

Definition 2.3.1 states the pointwise nonnegativity condition. Lemma 2.3.2 proves that predicate denotations preserve it. Theorem 2.3.4 is the soundness-and-completeness theorem, and Remark 2.3.5 identifies its load-bearing arithmetic facts:

$$\begin{aligned} a \geq 0, b \geq 0 &\Rightarrow (a + b = 0 \Leftrightarrow a = 0 \wedge b = 0), \\ ab = 0 &\Leftrightarrow a = 0 \vee b = 0. \end{aligned}$$

Thus the theorem is not a theorem that arbitrary field-valued residuals may be combined by the same operations. It is an exact theorem about zero sets under a protected nonnegative, characteristic-zero interpretation.

We machine-checked this algebraic kernel, rather than re-formalizing every matrix and syntactic feature of the source paper. In `FOLArithmetizationCorrected.lean`, the structure `ZeroTrueLaws` makes the noncancellation and no-zero-divisor obligations explicit. `PositiveFormula.residual_eq_zero_iff` proves exact compilation for the positive Boolean fragment; `PositiveFormula.residual_all_eq_zero_iff` and

`PositiveFormula.residual_any_eq_zero_iff` prove the corresponding finite quantifier folds. The instance

`nonnegativeRationalLaws` discharges precisely Gabbay’s nonnegative-rational obligations. This scope distinction matters: our Lean theorem validates the algebraic core used here, not every later construction in ePrint 2024/954.

2.2 Why the naive finite-field port fails

A finite field has no positivity order compatible with its ring operations, and nonzero summands can cancel.

Gabbay explicitly records this limitation in Remark 6.3.3 (“Why Q?”): the zero-sum property used by addition-as-conjunction is lost in a prime field. The proposed cryptographic route is to establish the polynomial equality over

$\mathbb{Q}[X]$ first and only then map that equality into a suitably large prime field. It is not a direct finite-field truth semantics, and the rational proof does not transfer unchanged.

The failure already occurs in BabyBear, the prime field with $p = 2,013,265,921$. Let $r = 1,728,404,513$. Lean verifies

$$r^2 = -1 \bmod p, \quad 1 \neq 0, \quad r^2 \neq 0, \quad 1 + r^2 = 0.$$

Taking squared equality residuals gives the concrete false conjunction

$$R(0 = 1) = 1, \quad R(0 = r) = r^2 = -1, \quad R((0 = 1) \wedge (0 = r)) = 0.$$

Both atoms are false under zero-means-true, yet their sum accepts. The exact certificate is

`FOLArithmetizationCounterexamples.babyBear_false_and_false_accepted`; `babyBear_sqrt_neg_one` certifies the field identity, and `babyBear_false_forall_accepted` lifts the same pair to a two-point universal quantifier. This is not a low-probability collision: it is an explicit witness. Nor is $1 - R(\varphi)$ a general repair for negation before Booleanization; `babyBear_one_sub_is_not_general_negation` gives the residual $R(\varphi) = 2$, for which both $R(\varphi)$ and $1 - R(\varphi)$ are nonzero.

Two exact finite-field repairs are already formalized. First, compile in nonnegative integers and prove that the complete residual is strictly below the field modulus before casting. `zmod_natCast_eq_zero_iff_of_lt` is the no-wrap lemma, and `natural_residual_cast_sound` is the compiler theorem. Second, reify each arbitrary residual r as a Boolean false-bit b with inverse witness z :

$$r(1 - b) = 0, \quad rz = b.$$

The constraints force $b \in \{0, 1\}$ and $b = 0 \Leftrightarrow r = 0$, proved by `falseBit_eq_zero_iff`, `falseBit_is_bit`, and `exists_falseBitWitness`. Once Booleanized, exact connectives are

$$b_{\varphi \wedge \psi} = b_{\varphi} + b_{\psi} - b_{\varphi} b_{\psi}, \quad b_{\varphi \vee \psi} = b_{\varphi} b_{\psi}, \quad b_{\neg \varphi} = 1 - b_{\varphi}.$$

`BooleanFormula.falseBit_correct` proves, for every formula in this Boolean core, both that the output remains a bit and that it is zero exactly when the source formula holds. These repairs have real costs—static bound certificates in the first case, and witness equations plus Boolean connective gates in the second—so they must be included in any performance comparison.

2.3 The BitLogic connective reversal

The public BitLogic PDF states on p. 4 that conjunction maps to product and disjunction to sum. Section 7.1 (p. 10) repeats that map and additionally maps bounded universal quantification to product and existential quantification to sum. Under zero-means-true, this reverses the intended logic even in an integral domain:

$$P_{\varphi} P_{\psi} = 0 \Leftrightarrow P_{\varphi} = 0 \vee P_{\psi} = 0.$$

Accordingly, a product recognizes disjunction, not conjunction. The Lean theorem

`FOLArithmetizationCounterexamples.zero_of_mul_iff_or` proves this generic fact. The finite-list regressions `product_is_not_zero_true_forall` and `sum_is_not_zero_true_exists` show both quantifier errors: the product $0 \cdot 7$ accepts a purported universal despite one false point, while the sum $1 + (-1)$ accepts a purported existential despite having no zero residual.

The issue is visible in the swap relation printed in Section 4.1 (p. 6). Writing $q = f(k, s)$ for the quoted output, the PDF defines

$$P_{\text{swap}} = (A' - A + k)(B' - B - q)(AB - A'B')$$

and accepts when $P_{\text{swap}} = 0$. Because this is a product, any one satisfied factor bypasses the other two. For example, over the integers choose

$$A = B = 10, \quad k = q = 1, \quad A' = 9, \quad B' = 999.$$

Then

$$A' - A + k = 0, \quad B' - B - q = 988 \neq 0, \quad AB - A'B' = -8,891 \neq 0,$$

so the printed relation accepts while its price/output and constant-product conditions both fail. This calculation is checked by `published_swap_accepts_invalid_witness`. More generally,

`published_swap_first_factor_bypasses_rest` proves the bypass in every commutative ring, and

`published_swap_bypass_is_polynomial_identity` proves

$$P_{\text{swap}(A,B,A-k,B',k,q)} = 0$$

for all remaining inputs. Random evaluation cannot detect constraints erased by an identically zero factor; Schwartz–Zippel bounds a nonzero polynomial’s accidental zeros, not a relation that is already the zero polynomial after the witness substitution.

Construction	Connectives (zero is true)	Finite quantifiers	Established scope
Gabbay, Thm. 2.3.4	$\wedge \mapsto +; \vee \mapsto \times$	$\forall \mapsto \sum; \exists \mapsto \prod$	Exact for pointwise nonnegative $\mathbb{Q}[X]$ denotations (formal theorem above).
BitLogic PDF, pp. 4, 10	$\wedge \mapsto \times; \vee \mapsto +$	$\forall \mapsto \prod; \exists \mapsto \sum$	Fails the connective and quantifier counterexamples above.
No-wrap field repair	Gabbay map, then bounded cast	Gabbay folds, then bounded cast	Exact under a proved residual bound; Lean: <code>natural_residual_cast_sound</code> .
Boolean field repair	$\wedge \mapsto a + b - ab; \vee \mapsto ab$	Boolean folds	Exact over fields after atom reification; Lean: <code>BooleanFormula.falseBit_correct</code> .

The authors’ May 2026 CTB workshop slides make the implementation direction more concrete: they move through natural/integer semantics, bit decomposition, Booleanity constraints, and a Zinc integer AIR. The slides explicitly set range checks aside in the presented sketch and list direct finite-field arithmetization and a non-toy implementation as future work. This is useful convergence on the repair obligations above, not support for omitting them. Our construction program therefore treats bounds, Booleanity, and range evidence as first-class compiler outputs rather than prose side conditions.

2.4 Evidence boundary and constructive next step

As of 22 July 2026, the named public BitLogic repository’s pinned tree contains exactly one 14-page PDF. It contains no executable source, build manifest, compiler, witness generator, prover, verifier, release, parameter set, benchmark harness, proof corpus, or raw measurement. The PDF names the real, third-party Zinc/Zip proving stack, but the public search found no Modulus FOL-to-Zinc compiler, proof-format adapter, or verifier integration. This is a reproducible statement about the named public artifacts, not a claim about private or differently named work. Consistently, the whitepaper’s Phase I roadmap (Section 9.1, p. 13) lists implementing the compiler and verifier and benchmarking polynomial-evaluation costs as work for 2025–2026.

The public performance claim is also not one stable metric. The current site applies “10–100x” to cost and adds “1–3 second finality”; a project-supplied 13 November 2025 release gives proof-price ranges; the 20 October 2025 project history instead says 10x faster and 90% smaller; and a 1 December 2025 post says 10–100x smaller proofs. Cost, proof size, prover time, and finality are different quantities. We found no public workload corpus, proof-system and security parameters, comparison implementation, hardware manifest, harness, or raw samples that bind any of them to the advertised multiplier.

“Finality” itself needs a tier. The official Modulus glossary distinguishes a trusted state committed by its single trusted sequencer, a virtual state whose batches have reached L1, and a consolidated state proven on L1 by a ZKP. It attributes fast or instant finality to the trusted sequencer and one-transaction L2 blocks. Without identifying one of these states, the site’s 1–3 second figure is not a reproducible settlement or proof-latency measurement.

There is now a useful first-party disclosure boundary. In a 21 July 2026 public reply, Modulus wrote: “There are major ones. Just announcing tech breakthroughs without the open source code is generally frowned upon ...” The same reply explains that commercial and internal positioning precedes publication. We therefore do not infer that private work is absent, nor that it matches the PDF. The decisive claim in this report is narrower: the one published BitLogic relation is not Gabbay’s finite-field construction and fails its stated source logic. Therefore we report a specification-level counterexample, not a production exploit and not a measured performance result. A deployed implementation could use different equations; publishing its compiler, proof interface, and verifier would resolve the discrepancy. The shared opportunity is an open conformance suite containing the examples above, corrected no-wrap and Booleanized lowerings, conventional circuit baselines, and identical proof-system parameters. Soundness is the admission gate; only implementations that agree on the source truth table should enter a latency, size, finality, or cost chart.

SECTION 3

Corrected constructions: from matrices to searched proof plans

The corrected development is now a coherent compiler stack rather than a replacement connective table. It begins with Gabbay-style integer matrices and exact rational interpolation, proves that an independently defined source semantics agrees with the compiled polynomial, crosses into a finite field only through an explicit projection certificate, and reaches DREGG’s live descriptor relation. A second compiler handles arbitrary finite signatures. Above both sit checked optimization and certified representation search.

CONSTRUCTION RESULT There is a sound version of the matrix idea. Its invariant is not “first-order logic is automatically a finite-field circuit.” The invariant is: exact nonnegative rational semantics first; a proved prime and no-wrap boundary or an exact Boolean repair second; and a checked live relation last. Every transition now has a theorem, a refusal case, and a stated cost and public-binding boundary.

3.1 Construction map

Layer	Construction	Exact result	Lean artifact
Matrix semantics	integer tables, one-based rational Lagrange rows, bounded matrix quantifiers	compiled residual is zero iff the independently defined source formula holds	<code>GabbayMatrixSemantics</code>
Compiler agreement	positive-arity adapter between the semantics view and the proof-carrying compiler	row, term, formula, truth, residual, cost, projection certificate, and Skolem witness agree	<code>GabbayMatrixCompiler / Bridge</code>
Field boundary	reduced numerator and positive denominator plus prime/no-wrap certificate	field zero iff rational zero iff source truth; bad modulus is rejected	<code>GabbayFiniteFieldProjection</code>
Live Gabbay instance	two-row, three-column successor table in <code>DescriptorIR2</code>	17-column constructive private trace; 12 constraints; exact for the table carried by that trace, but not externally bound	<code>GabbayDescriptorIR2</code>
General finite FOL	public total functions, witness relation tables, equality, all Boolean connectives and bounded quantifiers	canonical trace accepts iff the finite model satisfies the sentence	<code>FiniteSignatureFOLDescriptorIR2</code>
Optimization	checked local identities, factoring, sharing, and nonempty-fold elimination	source meaning and field-graph acceptance preserved; symbolic cost cannot increase	<code>DirectLogicOptimizerCertificate</code>
Representation search	finite graph of exact presentation changes and per-formula plan enumeration	selected composite remains exact; winner is minimal in the explicit candidate list	<code>CertifiedRepresentationSearch</code>

3.2 Exact matrix semantics and its compiler

For a matrix with `rows` rows and `length` declared columns, `IntMatrix.rowPolynomial` is the unique rational Lagrange interpolant through the one-based nodes 1, ..., `length`. The two facts needed downstream are proved directly:

$$P_{r(j+1)} = M_{r,j}, \quad \text{degree}(P_r) < \text{length}.$$

Terms contain the distinguished index, rational constants, arithmetic, matrix length, and row lookup. Formula atoms are squared equalities. Conjunction and bounded universal quantification add nonnegative residuals; disjunction and bounded existence multiply them. `residual_nonnegative` is the load-bearing lemma that rules out additive cancellation over $\mathbb{Q}$. The central theorem, `residual_eq_zero_iff_holds`, proves for every valuation, rational index, and formula φ :

$$\rho_{M,x}(\varphi) = 0 \quad \text{iff} \quad M, x \models \varphi.$$

The quantifiers range over exactly the columns declared by the selected matrix. `quantifierExpansion_eq_length` records the exact number of body instantiations. Because each bounded quantifier closes its index, its compiled polynomial is constant in the outer index; `forall_degree_le_zero` and `exists_degree_le_zero` state that fact. These are degree theorems, not claims about prover latency.

`GabbayMatrixCompiler` independently defines its own signature, matrices, source satisfaction, term evaluation, and polynomial compiler. Its `eval_polynomial_eq_zero_iff` proves the same exact source/compiler relation.

`GabbayMatrixBridge` then prevents agreement-by-prose: under `PositiveArity`, it proves equality of row polynomials, term and formula polynomials, source truth, residuals, and exact expansion costs.

`ViewCertificate.toCompiler` transports matrix bounds, numerator and denominator bounds, and the cost equality into the canonical compiler certificate.

FORMAL ARTIFACT - matrix/Skolem specimen The checked specimen is an actual two-row, three-column integer table. Its first row is the input and its second row is the chosen successor output.

`successorTable_size` proves an exact six-cell table; `successorTable_row_degrees_exact` proves that both interpolants have degree one. Through the bridge, exhaustive universal checking costs exactly three equality atoms and two additions, and `successor_skolem_through_bridge` constructs the finite Skolem witness. The transported certificate discharges the $p = 17$ projection theorem without repeating the bound proof.

BOUNDARY The bridge's only semantic mismatch is explicit: the exploratory semantics type permits a zero-column matrix, while the compiler signature requires positive arity. This stack formalizes bounded finite matrix semantics. It is neither ordinary unbounded first-order quantification nor a succinctness theorem.

3.3 The field boundary is a certificate, not a cast

Let the exactly evaluated residual be the reduced rational $q = \frac{a}{b}$, with $b > 0$. `ProjectionCertificate q p` carries three facts:

$$p \text{ is prime, } |a| < p, \quad b < p.$$

`projectCleared` sends only a into $\frac{\mathbb{Z}}{p}\mathbb{Z}$; `projectRational` sends $\frac{a}{b}$ after proving that the denominator remains invertible. `projectCleared_eq_zero_iff` and `projectRational_eq_zero_iff` prove that either representation is zero in the field exactly when $q = 0$. Composing that result with matrix correctness gives

$$\text{projected_formula_zero_iff_holds} \text{ and } \text{projected_rational_formula_zero_iff_holds}.$$

The compiler emits the auditable floor

$$\text{modulusFloor}(q) = \max(|a|, b) + 1.$$

Any prime at least this floor constructs the certificate. `ProjectionCost` retains four distinct quantities: formula degree, numerator magnitude, normalized denominator, and required modulus floor. They are not collapsed into a context-free speedup number.

EXECUTABLE COUNTEREXAMPLE The false atom $0 = 5$ has exact residual 25. An unchecked projection to $\frac{\mathbb{Z}}{5}$ returns zero, so `badPrime_unchecked_accepts` exhibits the failure in executable form. `badPrime_certificate_rejected` proves that the corrected compiler refuses the same case because the required strict inequality $25 < 5$ is impossible. This is the smallest useful answer to “why not just evaluate the rational construction in a field?”

For cases where a useful no-wrap certificate is unavailable, the independent Boolean repair remains exact over every field. Atomic zero tests introduce a bit and an inverse witness; Boolean connectives then operate on constrained bits. `Formula.falseBit_isBit_and_correct` and `falseBit_eq_zero_iff` in the matrix compiler prove that this alternative supports the full matrix formula language without pretending that rational addition retained its semantics after reduction.

3.4 Two routes into the live DREGG relation

3.4.1 A fixed-shape private witness/semantics bridge

`GabbayDescriptorIR2` lowers a nontrivial matrix instance into the actual `EffectVmDescriptor2` grammar used by DREGG. Its source is a two-row, three-column graph of a unary successor function. The trace contains the six table entries, six denominator-cleared interpolation coefficients, three squared residuals, their numerator, and a denominator marker: exactly 17 columns.

For row values y_1, y_2, y_3 , the compiler stores the coefficients of $2P(X)$:

$$(6y_1 - 6y_2 + 2y_3, -5y_1 + 8y_2 - 3y_3, y_1 - 2y_2 + y_3).$$

This avoids witness-authored division inside the interpolation checks. Six interpolation identities, three squared-residual identities, one numerator identity, one denominator identity, and one acceptance identity give exactly 12 live constraints. The coefficient columns are nevertheless redundant for the current acceptance path: the residual and acceptance gates read the raw input and output entry columns directly. This instance therefore demonstrates a faithful fixed-shape witness/semantics bridge, not a performance advantage from interpolation. Under `LiveProjectionCertificate`, `trace_satisfied_iff_holds` proves a table-indexed equivalence: `traceOf table` satisfies the emitted BabyBear relation exactly when the source formula holds of that same `table`. The canonical successor trace is constructed, the constructed trace of a one-cell tamper is refused, and the known modulus-five projection cannot enter the compiler.

BOUNDARY The current descriptor is an existential private-witness statement, not an attestation of an externally named table. `descriptor_public_surface_empty` proves `piCount = 0`, `hashSites = []`, and `ranges = []`. `constructed_traces_publicly_indistinguishable` proves that every constructed table trace exposes the same empty public assignment. Most sharply, `accepting_trace_does_not_attest_external_table` constructs an accepting successor witness while a separately named tampered table is false; there is no contradiction because that external table is absent from `Satisfied2`. A public input or binding commitment is required before this relation can attest a claimed model. It is also not yet a generator for arbitrary matrix dimensions: that requires emitted interpolation, denominator management, range certificates, and cost accounting. IR-v2 fixes BabyBear; the witness does not choose its own field.

3.4.2 General finite-signature FOL

`FiniteSignatureFOLDescriptorIR2` takes the complementary route. A signature may contain any finite list of relation symbols with independent arities and any finite list of total function symbols with complete public

interpretations. Terms include constants, de Bruijn variables, and nested function application. Formulae include equality, relation application, every Boolean connective, and exhaustive finite universal and existential quantification.

For a q -element domain and relation arities a_r , the canonical witness-row width is proved exactly:

$$W = \sum_r q^{a_r}.$$

The row concatenates every relation table in symbol-major, tuple-major order; the descriptor contains $W + 1$ constraints. `fol_sound`, `fol_complete`, and `canonical_model_trace_iff` prove soundness, constructive completeness, and the exact canonical-trace equivalence. A checked `FOLLiveCertificate` binds the version, complete function-table signature, relation-column layout, encoded source formula, grounded Boolean source, and the live descriptor certificate. `checked_fol_sound` carries those bindings through to the original sentence.

FORMAL ARTIFACT - nested finite-FOL specimen The $q = 2$ specimen has a public unary `flip` function, one unary relation, one binary relation, and the sentence $\forall x. \exists y. R_1(\text{flip}(x), y) \wedge R_0(\text{flip}(\text{flip}(x)))$. Its exact row width is $2 + 4 = 6$ and its descriptor has seven constraints. A canonical model trace satisfies the live relation; replacing the certified layout by the empty list is rejected.

BOUNDARY Grounding is exhaustive and can grow exponentially with quantifier depth. Function tables are public compilation inputs in this construction; private functions require a lookup or RAM lowering. The theorem is an exact finite model compiler, not a succinct quantifier protocol.

3.5 Checked optimization with exact savings

Optimization is now proof-producing. `DirectLogicOptimizerCertificate.Rule` contains only audited identities, nonempty-fold elimination, factorization, and explicit fanout sharing. A local claim binds the proposed before term, after term, rule payload, and side condition. `checkLocal` rejects a changed endpoint or a false guard.

Composite certificates additionally check that the target of one step is the source of the next.

For every accepted certificate, the formal results are simultaneous:

- `Certificate.check_sound` preserves source truth;
- `Certificate.accepts_iff` preserves the exact field `BoolGraph` relation;
- `Certificate.check_cost_nonincrease` proves componentwise nonincrease of equations, multiplications, witnesses, maximum degree, and dense table cells;
- `optimize_checked` proves that the deterministic recursive optimizer always emits a valid certificate; and
- `optimize_accepts_iff` and `optimize_cost_nonincrease` expose the end-to-end semantic and cost contracts.

Checked specimen	Equations	Multiplications	Witnesses	What changed
shared common factor	42 -> 30	40 -> 29	25 -> 18	one duplicated two-gate factor compiled once
recursive certified optimizer	30 -> 13	26 -> 13	17 -> 8	identities removed, then common factor shared

Both targets retain local degree two. The optimizer's hostile specimens reject a tampered target, an empty-fanout application, and a broken certificate boundary. These are counts in the abstract materialized `BoolGraph` ledger.

The current live DescriptorIR2 compiler instead uses atom Booleanity gates and nested `WindowExpr` syntax without the same intermediate columns, so these figures do not establish a saving in the current live prover. They are neither prover-time measurements nor an end-to-end benchmark.

3.6 Exact presentations and finite search

`CertifiedPresentationChange.Presentation` packages source meaning, evidence, acceptance, and a resource vector. An exact `Change P Q` maps evidence in both semantic directions and carries a compositional resource bound. Identity and composition form a genuine category, so a compiler can change representation without dissolving its proof obligation.

The concrete nodes are:

- proof-carrying natural no-wrap residuals;
- exact finite-field Boolean graphs; and
- hybrid evidence selecting either representation independently at each positive subformula.

`HybridEvidence.accepts_iff` proves every such mixture exact. The direct no-wrap-to-hybrid arrow agrees observationally with the route through a Boolean graph. `CertifiedRepresentationSearch` lifts these nodes to a finite graph of typed exact edges. Paths compose to exact changes, and their symbolic overhead composes with them. `shortestEnumerated_minimal` proves that the selected path has minimum scalar score among the caller-supplied candidates; `shortestEnumerated_composite_exact` proves that selection cannot change acceptance. The static specimen makes the selection logic concrete. In $\frac{\mathbb{Z}}{5}\mathbb{Z}$, two true atoms have actual residual zero but public upper bound three. The whole conjunction has static upper bound six and is therefore refused as one no-wrap node; each atom remains locally safe. The selected plan splits at the root and uses no-wrap leaves. It is proved accepting and minimal in the enumeration, with symbolic score 0 versus 23 for the all-Boolean graph.

BOUNDARY Search minimality is relative to an explicit finite candidate list and to the transparent score $E + M + W + D + T$; it is not global compiler optimality or a latency model. In the static specimen, score zero means that atomic no-wrap skeletons cost zero in this ledger and split acceptance is represented by meta-level conjunction. A concrete backend must materialize and charge any glue gates it needs. The theorem proves exact plan selection; it does not turn uncharged glue into free computation.

WHAT HAS BEEN RECOVERED The defensible performance thesis is local and certified: exploit a compact residual exactly where a range proof makes it sound; use Boolean zero tests where it does not; share repeated structure; and search only among meaning-preserving changes of presentation. The checked specimens show substantial symbolic reductions, but no theorem here implies a universal 10-100x end-to-end speedup or chain finality.

SECTION 4

Higher-order logic and the categorical boundary

There is no single object called “the higher-order arithmetization.” Four different constructions are exact, and they answer four different questions.

- Finite extensional HOL has a complete polynomial semantics, but arrow values are dense tables.
- Family-indexed finite-field hom codes exist exactly for locally finite categories; one-hot composition is quadratic.
- A typed STLC compiler targets shared nets without tabulating functions, and the source and target are genuine cartesian closed categories connected by a strong-CCC functor.

- Every small CCC embeds fully faithfully in a presheaf CCC by Yoneda, preserving the chosen exponentials up to explicit isomorphism. This is the general semantic answer, not a finite-field encoding.

The separation matters. The first two results are arithmetizations. The third is an intensional compiler with checkable local evidence. The fourth is a categorical embedding. Conflating them would turn a collection of precise theorems into one false performance claim.

FORMAL ARTIFACT - FOUR EXACT CONSTRUCTIONS `FiniteHOLFormulaArithmetization` proves the dense finite model; `FinitePolynomialCategoryClassification` proves the locally finite classification; `IntensionalCCCCategory` proves the shared-net CCC and its strong-CCC compiler; and `SmallCCCYonedaPresheaf` proves the arbitrary-small-CCC semantic embedding. None is advertised as a succinct arithmetization of every cartesian closed category.

4.1 Finite extensional HOL: complete, exact, and dense

Fix a finite field F of cardinality q . For a finite coordinate type σ , write F^σ for $\sigma \rightarrow F$. Every function from one finite field power to another is represented coordinatewise by a polynomial function. For one scalar output, the explicit interpolation formula is

$$\delta_{a(x)} = \prod_{i \in \sigma} (1 - (x_i - a_i)^{q-1}), \quad p_{f(X)} = \sum_{a \in F^\sigma} f(a) \delta_{a(X)}.$$

Fermat's finite-field law gives $\delta_{a(a)} = 1$ and $\delta_{a(x)} = 0$ for $x \neq a$, so $p_{f(x)} = f(x)$ at every field point.

`eval_interpolate` and `eval_interpolateVector` prove the scalar and vector forms; the corresponding surjectivity theorems state that every extensional map has such a representative. These are polynomial **functions**. Formal polynomials are identified by their evaluation on F : $X^q - X$ is generally nonzero as syntax and zero as a function.

The finite HOL construction uses types generated by unit, scalar, product, and arrow. A type A has a finite coordinate type C_A and value space $[A] = F^{C_A}$. An arrow has coordinates

$$C_{A \rightarrow B} = (F^{C_A}) \times C_B,$$

so an arrow value is its complete input-output table. Lambda fills the table; application reads it. The intrinsically typed term language validates product beta, function beta, and extensional eta. Its formula language adds extensional equality, all Boolean connectives, and universal and existential quantification at every generated type. In particular, quantification at arrow type really ranges over every table, rather than over a sample of functions. A typed environment is flattened losslessly by `environmentEquiv`. Every term coordinate has a polynomial

`termPolynomial`, with lambda and application boundaries proved exact. Every formula φ has a

`formulaPolynomial` that is Boolean on the entire ambient field power and satisfies

$$\text{eval}(p_\varphi, x) = 0 \quad \text{iff} \quad x \models \varphi.$$

The headline theorems are `eval_formulaPolynomial_point_isBit` and

`eval_formulaPolynomial_point_eq_zero_iff`; the forall, exists, and arrow-forall specializations are proved separately. This is a complete formal arithmetization of the defined finite extensional equality-HOL. It is noncomputable truth-table interpolation, not an extracted optimizing compiler.

4.1.1 Exact exponential cost and the fixed-model obstruction

If $|\sigma| = n$ and $|\tau| = m$, then

$$|(F^\sigma \rightarrow F^\tau)| = (q^m)^{q^n} = q^{mq^n}.$$

The exponential object therefore has exactly mq^n field coordinates and q^{mq^n} points. `card_expCoord`, `card_tableObject`, and `card_arrowQuantifierDomain` prove those counts. The blowup is the size of the represented function space, not slack in the analysis.

This also supplies the no-go theorem for one fixed finite extensional carrier. Any faithfully encoded hom-set must have at most q^{mq^n} elements (`finite_hom_bound`), and an infinite hom-set cannot inject into it (`no_injective_infinite_hom`). The concrete witness is the CCC `Type : already Nat → Bool` is infinite, so `no_injective_nat_predicates` rules out a faithful encoding into any fixed finite table space. An effective faithful encoding would also reflect decidable equality from the finite target.

BOUNDARY Finite extensional HOL is solved positively and exactly, but its higher-order values are complete tables. It cannot be promoted to an arithmetization of arbitrary CCCs by changing the prose around the theorem.

4.2 Locally finite categories: the exact indexed classification

There is a broader finite-field theorem, but its indexing is load-bearing. An `IndexedHomPresentation` chooses a finite coordinate type and an injective field vector code separately for every ordered pair of objects (A, B) . The theorem

indexed finite-field hom presentation exists iff every $\text{Hom}(A, B)$ is finite

is `indexedPresentation_nonempty_iff_locallyFinite`.

For the reverse direction, `oneHotPresentation` takes the hom-type itself as the coordinate set. On valid one-hot inputs, composition has output coordinate

$$\text{Comp}_{h(x,y)} = \sum_{f:A \rightarrow B} \sum_{g:B \rightarrow D} [g \circ f = h] x_f y_g.$$

This is a sum of bilinear monomials, so its total degree is at most two; `eval_compositionPolynomial` proves exactness. Identity is a constant one-hot vector, and code equality reflects equality of source morphisms. This result does **not** place the whole category inside one finite semantic object. The coordinate type varies with (A, B) ; an infinite locally finite category may need infinitely many such presentations; and product or exponential preservation is additional data. Nor is the one-hot representation succinct. It classifies a family-indexed arithmetization of hom-sets and composition, exactly and no more.

4.3 The intensional route: a shared-net CCC

Dense tables are not the only semantics for higher-order programs. The intensional development instead starts from intrinsically typed STLC with unit, natural numbers, products, arrows, and explicit source `let1`. Its target `Net` language adds a sharing binder. Beta compilation performs the local transformation

$$(\lambda x.b)a \longrightarrow \text{share } a \text{ as } x \text{ in } b,$$

so the argument is represented once and referenced wherever the variable occurs. `compile_denote`, `localStep_denote`, and `trace_denote` prove source compilation and every typed local trace denotationally exact. Product beta and extensional eta have their own local certificates.

The quantitative specimen binds an eight-operation computation once and uses it twice. Its shared target contains 8 work nodes and 13 total nodes; literal copying contains 16 work nodes and 19 total nodes. Beta costs one local rewrite. The shared cost is independent of q , while the matching one-input, two-output extensional table has $2q$ coordinates; `shared_smaller_than_table_coordinates` proves the shared graph smaller from $q \geq 7$. This is a checked structural comparison, not a prover latency benchmark.

4.3.1 A genuine category, not just beta examples

`IntensionalCCCCategory` closes the earlier gap. A shared-net arrow $A \rightarrow B$ is a semantic function equipped with a closed typed-net representative. Equality is equality of the represented functions. `ofNet_eq_iff` proves that two concrete nets map to the same arrow exactly when their denotations agree, and `ofNet_surjective` proves that every arrow in the category has a net representative.

Identity, composition, terminal arrows, pairing, evaluation, curry, and uncurry all have explicit closed-net representatives. The category laws and the terminal, product, beta, eta, and exponential universal equations are collected by `Shared.laws`. Exponentials are the typed intensional function objects; no finite table is materialized. The source STLC quotient/image is constructed as a second genuine CCC. The structural compiler induces `compilerFunctor` from the source category to the shared-net category. `Compiler.strongCCC` proves preservation of:

- identities and composition;
- the terminal object and terminal arrows;
- product objects, projections, and pairing; and
- exponential objects, evaluation, currying, and uncurrying.

Thus the higher-order compiler now has the categorical theorem one actually wants for its formalized fragment. This is stronger than saying that some beta redexes evaluate correctly. It is still not the universal property of the free CCC, an embedding of every CCC, or a polynomial proof system.

4.4 From typed reductions to checked interaction receipts

The intensional proof objects connect to DREGG's existing executable interaction-net checker.

`IntensionalCCCInteractionBridge` translates each typed local step into a receipt containing a distinct root-rule tag and the complete outer-to-inner address of the interaction. Beta, eta, first projection, and second projection have distinct root tags; congruence paths distinguish, among other cases, the value and body sides of a sharing binder.

`stepReceipt_checks` proves that every honest translation is accepted by the fail-closed Boolean checker.

`translatedStep_checked_and_source_sound` deliberately returns two separate facts:

receipt checker accepts $\wedge$ the original typed denotation is preserved.

This separation prevents a Boolean marker graph from being misrepresented as the higher-order value itself. Traces translate to checked batches, and hostile tests reject a wrong location and the wrong side of a sharing binder.

The bridge also fixes a proposed row budget: a step at address depth d occupies exactly $d + 1$ rows and $4(d + 1)$ cells. `constraint_rows_exact` and `constraint_cells_exact` prove those counts. The budget is an ABI for a later DescriptorIR2 emitter; this module does not claim that the rows have already been committed by a cryptographic IOP.

4.5 Arbitrary small CCCs: Yoneda is the semantic generalization

For a genuinely small cartesian closed category C , the Yoneda functor

$$y : C \rightarrow (C^{\text{op}} \rightarrow \text{Type})$$

is fully faithful. It preserves terminal objects and binary products because it preserves limits. More importantly, the formalization constructs the pointwise equivalence and presheaf isomorphism

$$y(B^A) \text{ isomorphic to } (yB)^{yA}.$$

The forward map uncurries an arrow into B^A and composes it with product pairing; the inverse evaluates a natural family at the two projections and curries. The proofs are `representableExponentialEquiv`, `representableExponentialIso`, and `smallCCCyonedaExponentialIso`.

This is the clean answer to the arbitrary-CCC challenge at the level of semantics: every small CCC embeds fully faithfully in a presheaf topos while retaining the chosen cartesian closed structure up to the exhibited isomorphisms. There is no finiteness hypothesis. Accordingly, there is also no finite-field coordinate bound, polynomial compiler, succinctness theorem, or cryptographic verifier hidden in the result. Arithmetic must be introduced later for selected finite observations or receipts, with its own range, binding, and cost proofs.

4.6 The presentation category correction

Certified changes of representation form a useful global category, but the global category is **not** cartesian closed. Each object in `CertifiedPresentationChange` carries its own source predicate, and every arrow proves equivalence of source meaning. A terminal object would therefore have to be meaning-equivalent to both truth and falsity. `no_weakTerminal` refutes even that existence condition, and `no_global_cartesian_structure` records the consequence.

The obvious attempt to use logical conjunction as a categorical product also fails. A first projection from $P \wedge Q$ to P would, under meaning-equivalent arrows, force $P \rightarrow Q$; `truth_and_falsity_has_no_firstProjection` gives the concrete counterexample. Logical conjunction and categorical product cannot simply be identified in the global presentation category.

The correct positive structure lives over a fixed meaning. In a fixed-meaning, zero-resource fibre, objects are evidence families and arrows are pointwise evidence transformers. Terminal evidence is `Unit`, product evidence is a pair, and exponential evidence is a pointwise function from one evidence type to another.

`EvidenceFamily.laws` proves terminality, product beta and eta, and exponential beta and eta. `Hom.toChange` embeds each fibre arrow as an exact global presentation change with the same meaning and zero bookkeeping cost. Products here combine two evidence representations of **the same predicate**; they are not conjunction of unrelated predicates. `Intensional.representedArrow` shows how closed shared nets can inhabit these evidence families without becoming dense function tables.

4.6.1 What has, and has not, been arithmetized

Construction	Exact result	Boundary
Finite extensional HOL	Every defined formula has an exact Boolean polynomial.	Complete arrow tables; finite carrier; dense quantification.
Locally finite hom presentation	Faithful one-hot hom codes with quadratic composition.	Family-indexed; no automatic product or exponential preservation.
STLC to shared nets	Genuine source and target CCCs; strong-CCC compiler; checked local receipts.	One generated intensional fragment; not a field encoding or arbitrary-CCC embedding.
Yoneda	Fully faithful embedding of every small CCC; terminal, products, and exponentials preserved.	Presheaf semantics, generally infinite; no succinct arithmetic backend.
Fixed-meaning evidence fibre	Pointwise CCC of evidence transformers.	The global category is not cartesian; zero-resource fibre laws are not cost laws.

An arithmetization of an arbitrary CCC would still require a computable assignment of objects and arrows to an arithmetic backend, a stated and proved equality notion, faithfulness, preservation of terminal objects, products, and exponentials, and honest resource bounds for composition, evaluation, curry, and equality checking. The present development supplies exact pieces of that map and rules out the naive fixed finite-table version. It does not pretend the remaining engineering and cryptographic obligations are already discharged.

SECTION 5

Proof objects beyond polynomials: realizability, descent, and transcript soundness

The corrected arithmetic constructions answer how to present a chosen judgment to a chosen algebraic backend. They do not make first-order syntax, polynomial syntax, or a finite field the primitive notion of proof. A stronger architecture starts with meaning, chooses evidence, gives that evidence a local agreement structure, and only then binds and samples it. Each transition needs its own theorem.

Layer	Mathematical question	Formal construction here	Deployment obligation
Meaning	What does the judgment assert?	finite semantics; realizability predicates; resource-sensitive denotations	application-specific atoms, effects, and observation policy
Evidence	What witnesses the judgment?	PCA realizers; assembly morphisms; interaction traces; presentation changes	a complete source-to-evidence compiler
Agreement	When do local views determine one global object?	exact descent; robust-soundness sockets; explicit finite testers	a code-specific distance and local-testability theorem
Transcript	How can hidden checks miss an obstruction?	commit-before-query games; exact miss counts; BabyBear query-bias bound	OOD and low-degree reductions with concrete parameters
Binding	Why can no response rewrite committed evidence?	an explicit point-binding premise	a commitment or PCS security reduction and Fiat–Shamir analysis

judgment $\rightarrow$ evidence $\rightarrow$ local views $\rightarrow$ hidden queries $\rightarrow$ bound transcript.

The categorical semantics explains what evidence and compatible restriction mean. The probability game explains what a sampled verifier can miss. The cryptographic backend explains why the prover cannot change an unseen view after learning the query. None of those three explanations subsumes the other two.

5.1 Realizability is a semantics of evidence, not a field trick

The formal base is a relational partial combinatory algebra. `PCA.App  $f \times y$` says that applying f to x is defined with unique result y ; the K and S laws are stated directly over that relation. The derived composition program is exact: `PCA.app_compose_iff` characterizes precisely the two consecutive applications it performs. An indexed predicate over I is a family $I \rightarrow \text{Set}(A)$, and an entailment is witnessed by one realizer that transforms every member at every index. `entails_refl`, `entails_trans`, and `entails_reindex` prove the indexed preorder and strict substitution action.

Chosen computable pairing, sums, and currying give the expected Heyting operations. The formal universal properties are `entails_conj_iff`, `disj_entails_iff`, and `entails_imp_iff`; truth, falsity, and preservation

under reindexing are proved alongside them. The construction does not assume that sum tags are injective or that application is globally total.

There is a useful finiteness warning. Under the usual nontrivial definition, a PCA cannot be a finite algebra. The Lean development includes `PCA.unitPCA`, a collapsed one-point inhabitant of the law interface, but does not advertise it as a computational realizability universe. Terwijn surveys the standard infinitude boundary in *Computability in Partial Combinatory Algebras*. A finite-field backend should encode a finite execution trace or certificate about a nontrivial PCA, not pretend to put the entire PCA into a small field carrier.

5.1.1 Quantifiers and the empty-fiber test

Existential quantification is ordinary union over a map fiber and is a left adjoint for every map

(`existsAlong_adjunction`). Ordinary intersection gives the direct right adjoint only for surjections

(`forallAlong_adjunction_of_surjective`). At an empty fiber, a pointwise premise supplies no uniform tracker and so cannot justify a defined application.

`forallReal` repairs exactly that failure. Its evidence is a suspended pair (e, x) ; the application ex is demanded at each point that actually lies over the target index and is not demanded for an empty fiber. Consequently

`forallReal_adjunction` proves the right-adjoint law for every set map, while `existsReal_adjunction` gives the matching arbitrary-map left adjoint. `existsReal_beckChevalley` and `forallReal_beckChevalley` prove literal Beck–Chevalley equalities for weak pullback squares. The regression theorem

`forallReal_empty_equiprovable_truth` confirms that an empty-fiber universal is equiprovable with truth.

The uniform-family presentation packages these results without changing their meaning. `pcaUFam_tracks_iff`

and `pcaUFam_entails_iff` identify it exactly with the PCA doctrine. `pcaUFamTriposLaws` collects the Heyting preorder, strict reindexing, both arbitrary-map adjoints, Beck–Chevalley, exact generic classification, and noncollapse. In particular, `pcaUFam_truth_not_entails_falsity` proves consistency already at a singleton index.

FORMAL ARTIFACT - tripos boundary A Set-indexed realizability tripos-law package has been proved. The associated elementary topos has not. The development does not yet construct the exact completion, a subobject classifier, or an equivalence between that completion and the topos associated with `pcaUFamTriposLaws`. Pitts’ tripos-to-topos account describes the classical route citation is not a substitute for the missing Lean construction.

5.1.2 Assemblies make the program boundary concrete

`Assembly` pairs a semantic carrier with a nonempty realizability relation. An assembly morphism is an actual semantic function tracked uniformly by one PCA element. `categoryAssembly` proves genuine category laws: the derived identity combinator tracks identity, the exact composition program tracks composition, and arrow equality is equality of the semantic functions.

This category already has useful finite-limit structure. The terminal assembly has its unique incoming arrow. A chosen PCA pairing supplies binary products, with `pair_fst`, `pair_snd`, and `pair_unique` proving the universal property. Equalizers inherit the source realizers, and `equalizerLift_1` plus `equalizerLift_unique` prove factorization and uniqueness without changing the original tracker.

Modest assemblies induce partial equivalence relations on realizers. `tracker_respects_realizerPER` proves that every uniform tracker transports those PERs and returns both output realizers and their exact PCA applications. At the compiler boundary, `ProgramCertificate` contains input and output representations, a PCA program, and a theorem that the program produces the encoded result. `ProgramCertificate.hom` turns it into an assembly

morphism, while `ProgramCertificate.comp_hom` proves that certificate composition agrees with categorical composition. An extensional polynomial function alone supplies none of that executable evidence.

5.2 Presentations form fibers, not one global CCC

The resource-aware `Presentation` interface records source meaning, evidence, acceptance, and a five-axis backend cost. An exact `Change` maps evidence, preserves meaning and acceptance in both directions, and carries a compositional cost bound. These changes form a category. This is the right place to compose residual, Boolean-graph, shared-net, proof, and FHE representations of the same judgment.

The global category is deliberately not cartesian closed. Its objects may carry different meanings, while an exact arrow requires those meanings to be equivalent. `no_weakTerminal` proves that no object can receive arrows from both the true and false presentations; hence `no_global_cartesian_structure`. Likewise, the obvious presentation of logical conjunction generally has no categorical projections: `firstProjection_forces_right` and

`secondProjection_forces_left` show that such projections would force implications between the two meanings. The positive replacement is fibred. Over one fixed meaning, `EvidenceFamily` objects vary only the evidence type. Pointwise evidence transformers form a genuine cartesian closed category: unit evidence is terminal, paired evidence is product, and evidence transformers form exponentials. `EvidenceFamily.laws` packages product beta/eta and exponential beta/eta. This result complements the extensional finite CCC boundary in Section 4 without changing it: the exponential here is a space of evidence transformers inside one semantic fiber, not a finite-field table of every source morphism and not a succinctness theorem.

5.3 Exact descent: compatible local evidence glues uniquely

`ProofObjectDescent` isolates the backend-neutral local-to-global theorem. A `DataCover` includes a computational locator. `Matching` says local sections agree on every pairwise overlap, and `glue` chooses the located local value pointwise. The central theorem `matching_iff_existsUnique_amalgamation` proves

$$\text{all overlaps match} \iff \exists!g, \text{ every local view restricts from } g.$$

`incompatible_no_amalgamation` proves the refusal direction. `verifiedEvidence_descends` applies the same theorem to subtype-valued evidence $\{w : W_x \mid \text{Checks}(x, w)\}$, so the glued global witness retains its local verification proofs. This theorem contains no polynomial, field, hash, commitment, random oracle, or FRI premise.

Exact descent checks every relevant overlap. `RobustProofObjectDescent` exposes what sampled checking additionally requires:

- `WeightedQueryModel` permits an arbitrary finite weighted query distribution;
- `RejectLowerSoundness` and `DistanceUpperSoundness` carry explicit, possibly nonlinear robustness moduli;
- `FoldChainSoundness` separates distance preservation from weighted exceptional challenges; and
- `ProximityGap` bounds challenges that transform a far word into a close word.

These are theorem sockets, not assumed code theorems. Their soundness, exceptional weight, and gap fields must be instantiated for the deployed graph, sheaf, HDX, or FRI code. Two finite instances show that the interfaces are nonvacuous. `FinitePatchCode.reconstruct_codeword` reconstructs the unique global codeword from matching valid patches. The three-patch `WeightedStar` theorem `rejection_eq_hubDistance` gives exact equality between rejected weight and distance to the hub-selected codeword; its one-error example proves coefficient one sharp and refutes a false factor-two bound.

5.4 Hidden-query soundness: price the obstruction that was missed

Exact gluing does not imply that spot checks deterministically extract a global object. The corrected statement is probabilistic and keeps the committed family visible. If a finite section family has no amalgamation,

`badEdges_nonempty_of_no_amalgamation` proves that its obstruction set is nonempty. For a query space of size Q , an obstruction set of size B , and k independent with-replacement queries, `missSamples_card_sub` proves the exact count

$$\text{misses} = (Q - B)^k, \quad \text{all samples} = Q^k.$$

Thus the exact uniform miss probability is $\left(1 - \frac{B}{Q}\right)^k$. A bad committed object can pass on precisely this event; acceptance alone does not yield a total column.

`QueryOpeningScheme` separates committed family, digest, view, opening relation, and opening proof.

`PointBinding` states the anti-rewriting property actually consumed by the verifier game: after committing, any accepted opening at a query equals that query's committed view. The responder may otherwise be adaptive.

`acceptsSample_misses` proves that a bound responder accepted on every sampled query must have missed every obstruction, and `acceptedSamples_card_le` transfers the exact miss count to an acceptance bound.

`invalid_gluing_acceptance_bound` specializes the statement to overlap inconsistency. In the transparent reference game, two of four overlap edges are bad; two queries miss them in exactly four of sixteen samples, and the accepted-sample count is at most four.

FORMAL ARTIFACT - query-game correction The proved conclusion is a finite event inclusion and count, not deterministic extraction. Point binding prevents post-query rewriting; it does not make hidden queries hit every obstruction. A computational deployment must add the adversary's advantage in breaking its concrete commitment's binding game.

5.5 Full transcript composition, including BabyBear bias

`RobustDescentTranscriptGame` makes the protocol order explicit:

commit family $\rightarrow$ sample transcript seed $\rightarrow$ derive challenges and queries $\rightarrow$ open queried views.

A `TranscriptResponder` may inspect the entire revealed seed and the query coordinate. Point binding still forces every accepted response to equal the precommitted view (`locallyAccepts_misses`). The protocol-specific

`AcceptanceReduction` then has one clear job: prove that full acceptance implies at least one of three events:

1. all bound local openings pass;
2. an OOD algebraic exception occurs; or
3. a low-degree or folding exception occurs.

`acceptSeeds_subset_exceptional_union_miss` proves the corresponding event inclusion. `acceptProb_le_union` proves the exact finite union bound. It requires no independence between the OOD and low-degree events. The query sampler is also modeled rather than idealized. A squeeze value in $\text{Fin}(N)$ is reduced modulo a query-domain size m . If the obstruction density is at least δ , `mod_miss_prob_le` gives the conservative term

$$\left((1 - \delta) + \frac{m}{N}\right)^k.$$

When $N \bmod m = 1$, only one residue class is heavy. The sharp theorem `mod_miss_prob_le_sharp` improves the term to

$$\left((1 - \delta) + \frac{\delta}{N}\right)^k.$$

`biased_transcript_soundness_sharp_extra` allows every non-query transcript coordinate - OOD points, folding challenges, batching challenges, and future coordinates - to remain in an arbitrary finite `Extra` factor. The exceptional predicates and adaptive responder may inspect all of it. The query-marginal probability is unchanged because the uniform extra coordinates product out.

For BabyBear, $p = 2013265921$. For every realizable power-of-two domain $m = 2^\ell$ with $1 \leq \ell \leq 27$, the proved arithmetic fact is $p \bmod m = 1$. Therefore `babybear_transcript_soundness_sharp_extra` proves the complete bound

$$\Pr[\text{false acceptance}] \leq \varepsilon_{\text{OOD}} + \varepsilon_{\text{LDT}} + \left((1 - \delta) + \frac{\delta}{2013265921} \right)^k.$$

This is the exact sharp BabyBear expression formalized by the current game. The $\frac{\delta}{p}$ term is the modular-reduction defect; it is not silently rounded away and does not depend on trace height. The finite reference transcript proves that all three addends can be load-bearing: at $N = 5$, $m = 2$, $k = 1$, and $\delta = \frac{1}{2}$, the query term is $\frac{3}{5}$, the OOD and low-degree terms are each $\frac{1}{5}$, and the union bound is attained exactly.

BOUNDARY The BabyBear theorem is conditional on a fixed precommitted invalid object, point binding, the stated obstruction-density premise, and a protocol-specific acceptance reduction. It does not itself prove the deployed commitment binding, an OOD Schwartz–Zippel bound, a FRI low-degree theorem, random-oracle security, Fiat–Shamir soundness, or a total-column extraction theorem. Those premises must be instantiated and their error terms composed before quoting a bit-security number.

5.5.1 What topos theory contributes - and what it cannot contribute alone

Mechanism	It can establish	It cannot establish by itself
Realizability / tripos	meaning, uniform evidence transformation, logical adjoints, substitution	local testability, hiding, commitment binding, proof size
Assemblies / PERs	tracked semantic maps, executable representation boundaries, extensional quotients	a compiler program without a <code>ProgramCertificate</code>
Descent / sheaf condition	unique gluing from complete compatible local data	that a few sampled overlaps expose every inconsistency
Robust code theorem	distance versus rejected-query weight and proximity gaps	that an adaptive prover is bound to one word
Commitment / PCS	binding or hiding under a stated security game	semantic correctness of the committed proof object
FRI / graph IOPP	a quantitative low-degree or code-proximity test	the source logic semantics or commitment theorem

This separation leaves room for non-polynomial proof objects. First and Kaufman’s cosystolic expansion of sheaves on posets gives two-query locally testable codes [arXiv:2403.19388](#). Dinur, Liu, and Zhang’s HDX Tanner codes combine local Reed–Solomon views with a multiplication property [arXiv:2308.15563](#). Flowering and Cayley-graph IOPPs provide graph-indexed alternatives [arXiv:2501.14337](#) [arXiv:2508.10510](#). Each is a candidate

for a quantitative socket, not an imported theorem about DREGG's deployed parameters. FRI remains one established polynomial-code realization ICALP 2018, not the definition of proof-object descent.

5.6 Local interaction traces are proof objects too

The interaction route makes evidence a replayable list of local graph changes. `InteractionNetTrace` uses a finite typed Boolean port tree. An `ActiveRule` contracts one literal/gate interaction, and a one-hole `Context` identifies its location. `activeRule_preserves` proves every primitive rewrite; `contextualRule_preserves` lifts it under arbitrary contexts. `LocalCert.check` is executable and fail-closed. `checkTrace_sound` proves denotational preservation for accepted replays, while `checkTrace_reduces` reconstructs the finite reduction.

`LinearInteractionTrace` independently tracks resource kind, ticket, polarity, and explicit provenance. Its trace checker preserves the extensional per-ticket observation, total endpoint provenance, each ticket's count, and signed balance. The executable counterexamples reject cross-kind closing, a rule at the wrong location, and silent weakening. These traces are genuine proof objects that need not first be formulas or residual polynomials. They are not yet a succinct argument. The remaining path is concrete: compile a typed interaction trace to local patch constraints; prove exact descent of its denotation and resource invariant; instantiate a robust code/proximity theorem; then bind the encoded patches and apply the transcript game above.

CONSTRUCTION MAP Realizability supplies a semantics of evidence and quantification. Assemblies and certified presentations expose program and representation boundaries. Exact descent supplies the local-to-global law. Robust code theorems price inconsistency; the hidden-query and full-transcript games price missed obstructions and modular query bias. Commitments, FRI, graph IOPPs, and interaction traces are composable realizations of particular layers. No layer requires FOL, hashes, or polynomials to be the primitive notion of logic.

SECTION 6

From finite logic to a live proof and encrypted plan

The constructive result is now a pipeline, not merely a corrected polynomial identity. A finite-signature first-order sentence can be grounded into an exact Boolean relation, compiled into the live DREGG descriptor IR, accompanied by a canonical trace, checked against exact emitted bytes, optimized by replayable certificates, and related to an encrypted evaluation over the same opening. There are executable Rust, Python, Standard ML, HOL4, and CakeML artifacts at different assurance boundaries.

finite FOL → grounded Boolean relation → checked representation plan → DescriptorIR2 + proof
 same grounded source → shared opening → BFV/hybrid plan

This diagram is a map of proved and executable interfaces, not one monolithic end-to-end theorem. In particular, a Lean compiler proof, a Rust prover test, and a real BFV execution are three different kinds of evidence. The named boundaries below prevent one from being advertised as another.

6.1 General finite FOL reaches the live descriptor IR

`FiniteSignatureFOLDescriptorIR2` is the broadest verified source compiler in this study. A signature contains any finite list of total public function symbols and relation symbols of independently chosen finite arities. Terms contain constants, de Bruijn variables, and nested function applications. Formulae contain equality, relation application, negation, conjunction, disjunction, and exhaustive finite universal and existential quantification. For a domain of size q , a relation of arity a receives q^a canonical columns. The complete relation layout therefore has width

$$W = \sum_{r \in \text{relations}} q^{a_r}.$$

Public function tables are compiled by evaluation, while relation tables are the model-bearing trace data.

`compileDescriptorFOL` emits exactly W always-on bitness constraints and one always-on source-acceptance constraint. Quantifiers are grounded by exhaustive enumeration, so the construction is exact but can grow exponentially with quantifier depth. It is not presented as a succinct quantifier protocol.

FORMAL ARTIFACT - finite-signature live compiler `lower_correct` relates the direct finite-model semantics to the grounded Boolean source. `fol_sound` proves that every row of a satisfying canonical live trace satisfies the original sentence. `fol_complete` constructs a one-row live trace from every satisfying model, and `canonical_model_trace_iff` gives the exact equivalence. The fail-closed `FOLLiveCertificate` binds the domain and signature, complete public function tables, canonical relation layout, encoded sentence, grounded source, atom count, and exact DescriptorIR2 JSON. `checked_fol_sound` transports a checked certificate and a satisfying trace back to the source sentence.

The concrete nested specimen has a two-element domain, a public Boolean flip function, one unary relation, one binary relation, and the sentence

$$\forall x, \exists y, R_2(\text{flip}(x), y) \wedge R_1(\text{flip}(\text{flip}(x))).$$

Its canonical relation width is $2^1 + 2^2 = 6$ and its live descriptor has seven constraints. A certificate with an empty claimed layout is rejected. This specimen exercises functions, two relation arities, nested quantifiers, and exact live-byte binding in one kernel-checked construction.

The same target also contains a faithful fixed Gabbay matrix instance. `GabbayDescriptorIR2` compiles a two-row, three-entry Skolem graph using denominator-cleared Lagrange interpolation, squared logical residuals, an exact aggregate numerator, and a BabyBear no-wrap certificate. Its `trace_satisfied_iff_holds` theorem relates the actual live relation to the ordinary bounded source formula; successor data is accepted, while a tampered entry and the known unchecked modulus-five projection are refused. Arbitrary matrix length remains a separate symbolic compiler problem because it must emit and cost an interpolation algorithm rather than hide divisions in witness data. The fixed theorem constructs a trace from an externally named three-entry table, but the descriptor itself proves only an existential private-witness relation. `GabbayDescriptorIR2PublicBoundary` proves that its public-input, hash-site, and range surfaces are all empty and that all constructed tables have the same public assignment. It therefore cannot attest an independently named external table. Moreover, interpolation coefficients are consistency columns; the logical residual and final acceptance read the raw input/output cells. This specimen establishes a fixed-shape semantic relation, not an interpolation performance result.

6.2 Certificates optimize the relation without changing it

The important performance mechanism is not “FOL instead of circuits.” It is certified selection and sharing among equivalent presentations. `DirectLogicOptimizerCertificate` is an executable translation validator for a bounded Boolean fragment. Each local certificate contains redundant before and after terms plus a rule payload. The checker reconstructs both endpoints, checks the rule guard, checks every transitive boundary, and rejects tampered endpoints or disconnected certificate chains.

Accepted certificates preserve three meanings at once:

- source truth under every atom interpretation;
- the exact field `BoolGraph.Accepts` relation; and
- nonincrease of equations, multiplications, witnesses, and maximum degree.

The deterministic optimizer performs constant elimination, double-negation elimination, idempotence, Boolean factoring, and explicit fanout sharing. The factoring specimen has the following exact kernel-checked ledger:

Representation	Equations	Mults	Witnesses	Degree
Duplicated Boolean graph	30	26	17	2
Checked factored graph	13	13	8	2
Reduction	56.7%	50.0%	52.9%	none

Those percentages are exact symbolic reductions for one specimen, not prover latency ratios. They are nevertheless stronger than an optimizer’s assertion: `optimize_checked`, `optimize_holds_iff`, `optimize_accepts_iff`, and `optimize_cost_nonincrease` independently expose the certificate, semantics, field relation, and cost proof. This is an abstract materialized `BoolGraph` ledger, not the current live `DescriptorIR2` layout, so it proves no present live prover saving.

The standalone reference tool adds per-subformula representation search. `tools/direct-logic-reference` keeps a Pareto frontier of positive no-wrap residual and exact Boolean candidates and records every residual-to-Boolean or Boolean-to-residual boundary. Its independently rechecked mixed specimen has the lexicographic objective

Plan	Mults	Equations	Witnesses	Conversions	Nodes
Selected hybrid	5	9	6	1	5
All Boolean	9	11	6	0	4
All no-wrap	unsupported	-	-	-	-

The all-no-wrap plan is refused because the single-field bound cannot certify it. The hybrid artifact is canonical JSON with SHA-256 `540ad40d3c282b1294db7b62ab9ad80cf287486014aba9b054369b3a79fcdef3` ; the ordinary compiler artifact remains pinned independently at

`e1d19701afcbc58adf5c624dcf33ca323375a9f858e167c0d5f258c03a11f0cd` . `check` verifies the hash, recompiles from the embedded source, reconstructs the chosen plan and baselines, checks every primitive equation and bound, and then compares admitted plans with source semantics. The Python program is an executable specification and witness-plan checker, not a proof system.

6.3 One opening, proof and FHE

`ProofFheSharedOpening` closes a semantic triangle that the earlier report only proposed. A typed Boolean source carries its atom bound; one `Opening n` contains the raw integer word for every atom. Admission is integer-exact: every word must be literally zero or one, not merely congruent to a bit modulo a field.

The proof side reads the raw opening as a one-row `DescriptorIR2` trace. The FHE side reads the Boolean decoding of that same opening. For every commutative ring, `descriptorDenotation_eq_fheRing` proves equality between the descriptor source polynomial and the structurally lowered BFV Boolean program.

`proofAccepts_iff_fheOutput_one` then proves predicate-level agreement in BabyBear, while

`noncanonical_rejected` proves that an opening containing a non-bit is rejected by both interfaces.

`ProofFheDualCertificate` makes the bridge fail closed. Its versioned bundle binds the source, opening schema, exact `DescriptorIR2` JSON, exact BFV program, the BFV eight-axis operation ledger, and a length-framed canonical wire image. An accepted bundle guarantees backend denotation agreement for every canonical opening of the declared arity, not only the opening carried by the bundle. Unknown versions, trailing words, noncanonical openings, descriptor substitution, and BFV substitution are all explicit rejection theorems.

BOUNDARY The dual wire image is currently a mathematical `List Int` , not a proved Rust byte decoder. Ciphertext serialization, key ownership, leakage policy, threshold decryption, Rust-to-Lean refinement, and cryptographic security are outside these semantic theorems. The theorem says that both backends compute the same predicate on one admitted opening; it does not say that a deployed client supplied that opening correctly.

6.3.1 Certified mixed FHE plan

`CertifiedHybridProofFhe` goes beyond two identical Boolean lowerings. Its source consists of equality atoms over one raw opening. A conversion witness derives the exact equality-bit opening consumed by `DescriptorIR2`. The encrypted side may select, independently at each region, either a locally bounded zero-means-true residual or a one-means-true Boolean program. Every conversion between those conventions is present in the plan and its cost.

`Certificate.proof_iff_fhe_plan` proves that the live proof relation and mixed encrypted plan accept the same source predicate for every canonical raw opening. The concrete five-equality specimen deliberately makes the root residual inadmissible while leaving a four-equality subregion admissible:

Plan	Adds	Subs	Mults	Relins	Depth	Zero boundary
Certified mixed	5	6	6	6	2	1
All Boolean	10	10	9	9	4	0

Thus the mixed plan uses one third fewer ciphertext multiplications and half the symbolic multiplicative depth on this specimen. It also owes one private residual-zero boundary. Omitting that last column would make the comparison misleading.

BOUNDARY Three cost scopes must remain separate. The 30-to-13 optimizer result counts an abstract materialized `BoolGraph`, not current live `DescriptorIR2`. The theorem-side presentation search uses meta-level `HybridEvidence` glue, so a zero search score omits backend recombination and conversion. Finally, `boundaryZeroDecision` is a semantic counter: the 6-versus-9 hybrid ledger neither implements nor prices an encrypted residual-to-bit BFV zero test.

6.4 Executable backends

6.4.1 Rust finite-logic front end and live proof roundtrip

`circuit/src/direct_logic_frontend.rs` is an executable Rust compiler for finite Boolean and enumeration inputs, equality, explicit finite predicate tables, all Boolean connectives, and bounded quantifiers. Enumeration values use exact one-hot encodings; quantifiers are deterministically unrolled. Explicit limits reject an empty or unbounded domain, oversized predicate table, excessive quantifier expansion, unsupported source, and out-of-range input rather than silently changing the relation.

The compiler emits the deployed `EffectVmDescriptor2` structure and canonical JSON, reparses and structurally checks that JSON, and pins BLAKE3 digests of the source and descriptor. The focused suite contains nine tests covering source evaluation, descriptor equivalence, one-hot rejection, resource refusal, hash stability, and malformed inputs. Most importantly, `honest_compilation_proves_and_verifies_through_live_backend` sends a compiled one-row trace through the actual `prove_vm_descriptor2` and `verify_vm_descriptor2` APIs; the same prover refuses a row that falsifies the source formula.

This is the first live Rust prove/verify path for the construction. It is not yet a proof that the Rust compiler refines the Lean compiler, and one tiny roundtrip says nothing about production throughput or finality.

The path now also has an absolute release benchmark. It measures `LogicProgram` to `compile_logic_program` to `EffectVmDescriptor2` to one-row trace to `prove_vm_descriptor2` and `verify_vm_descriptor2`. The capture used an Apple M2 Max with 12 logical CPUs and 96 GiB RAM, five warmups, 30 proving samples, and 100 verification samples. The live BatchSTARK parameters were BabyBear, extension degree 4, FRI log blowup 6, 19 queries, and 16 query-PoW bits. All proofs had `degree_bits = 0`; proof sizes are the live SDK's postcard encoding.

Workload	Cols / gates	Prove median / p95	Verify median / p95	Proof bytes
<code>bool-eq</code>	1 / 2	2.219 / 4.572 ms	0.282 / 0.789 ms	9,262
<code>enum-eq-4</code>	4 / 6	6.911 / 9.829 ms	0.293 / 0.691 ms	9,342
<code>enum-eq-16</code>	16 / 18	4.497 / 7.140 ms	0.309 / 0.686 ms	9,721
<code>enum-eq-64</code>	64 / 66	6.728 / 8.596 ms	0.500 / 0.862 ms	10,924
<code>forall-4</code>	4 / 6	19.637 / 23.838 ms	0.361 / 0.794 ms	10,202
<code>forall-8</code>	8 / 10	8.828 / 10.630 ms	0.488 / 1.086 ms	11,165
<code>forall-16</code>	16 / 18	26.833 / 31.437 ms	0.800 / 1.695 ms	13,101

The proving medians are visibly nonmonotone. The shared development host had load averages 31.34 / 37.81 / 38.25, so the result supplies neither a scaling law nor a speedup comparison. Release proving excludes debug self-verification; the harness separately verified every warmup and retained proof. It does not measure the abstract optimizer, a conventional-circuit or Modulus baseline, zero knowledge, network inclusion, or finality.

Stable capture: docs/deos/artifacts/direct-logic-live-2026-07-22-m2-max . Summary SHA-256:

39a623da8ea21a03e69a6f420d5b49ce52762a9e4cbb9fefb21e7ae661db73 . Raw-log SHA-256:

66b37bf19bffe505b967b03fdf44a0cc8bde22ca232493417fe3350b9ef3b1e .

6.4.2 Independent HOL4 and CakeML checker boundary

`tools/direct-logic-cakeml` reimplements the certificate check independently of Lean. The version-1 Standard ML checker passed 15/15 golden and malformed fixtures. In HOL4, `check_bytes_sound` proves that accepted bytes decode under the known version to the canonical source/target pair and that their Boolean semantics agree for every environment. The theory has zero axioms in the recorded fresh audit, and `check_bytes_v_thm` is the proof-producing CakeML translation certificate.

The live version-2 checker embeds the complete length-delimited v1 certificate, a 32-bit atom bound, and exact canonical IR-v2 JSON. It reconstructs the Booleanity gates, acceptance polynomial, table and layout, then compares exact bytes. It caps the atom count at 256 and each section at one MiB. The live SML suite passed 11/11 fixtures, including exact agreement with Lean’s pinned demo JSON and rejection of version, bound, embedded-target, JSON, trailing, and truncation tampering. HOL4’s `check_live_bytes_sound` establishes the embedded v1 check, atom bound, exact descriptor bytes, and source/target semantic equivalence;

`check_live_bytes_v_thm` translates the live checker to CakeML.

BOUNDARY The checked and translated source is not yet counted here as a native verified checker binary. A machine-code claim requires the completed CakeML backend compilation plus a fresh theorem-tag and axiom audit of the live theories. The live checker byte-compares reconstructed JSON; it does not independently parse arbitrary descriptor JSON or prove the DREGG Rust verifier equivalent to the HOL relation.

6.4.3 Real BFV execution

`FheLogicBfvModel` proves the Boolean-ring identities and a no-wrap theorem for the nonnegative squared-equality residual

$$R = \sum_i (x_i - y_i)^2.$$

Under the declared input bounds and centered plaintext window, field zero is equivalent to all source equalities. For eight encrypted equalities, the residual interpreter has 8 multiplications, 8 relinearizations, and symbolic depth 1. The balanced all-Boolean interpreter has 15 multiplications, 15 relinearizations, and depth 4. The residual row excludes the required final private zero decision.

The matching `fhegg-fhe` path executes real BFV ciphertexts. One recorded warm release run on `persvati` (x86-64 Linux, AMD Ryzen AI 9 HX PRO 370), at ring degree 4096 and plaintext modulus 1,032,193, produced:

Executed workload	SIMD cases	BFV body	Mul/relin	Noise
Boolean policy	16	22.950435 ms	3 / 3	79 bits
Eight-equality residual	4	48.197263 ms	8 / 8	49 bits

The residual certificate checked $648 < 516096$. These rows are different programs with different SIMD occupancy, so their times do not define a speedup. They are single warm observations. The timed body excludes key generation, encryption, decryption, and the residual plan’s final private zero decision; the observed noise is not a proved noise or security bound. The focused release suite passed 3/3 tests, including refusal of a cancellation-capable bound.

6.5 Conformance before performance

`tools/direct-logic-bench` compares five relations against a common Boolean reference. A lane that fails semantics remains visible as a falsification artifact but is excluded from every performance ratio. The deterministic corpus contains the pinned adversarial and transaction examples plus 10,000 generated formulae, for 10,061 cases in total:

Lane	Gate	Supported	Unsupported	Mismatch
M-SPEC	fail	10,061	0	3,936
G-FIELD-NAIVE	fail	6,748	3,313	1
D-NOWRAP	pass	6,737	3,324	0
D-BOOLGRAPH	pass	10,061	0	0
C-AIR	pass	10,061	0	0

M-SPEC is an exact transcription of the public BitLogic connective, quantifier, and swap equations. G-FIELD-NAIVE separately tests the Gabbay-style positive residual after unsafe field reduction. D-NOWRAP is the corrected bounded positive route; unsupported means that its required partial-accumulator bound was not established. D-BOOLGRAPH is the exact inverse-witness Boolean graph. C-AIR is an independently written gate-by-gate arithmetic baseline. The harness emits environment and source hashes, every workload and gate result, a representation-labelled symbolic ledger, raw timing samples, bootstrap intervals, checksums, and an admitted-only summary. Its timing channel measures Python relation evaluation. It does not measure witness generation, proving, verification, proof bytes, FHE, consensus, data availability, finality, or monetary cost. In particular, an unsound equation does not become an optimization because it evaluates quickly.

6.6 What would test the claimed performance numbers

The verified constructions justify an end-to-end benchmark campaign; they do not pre-decide its outcome. A credible 10-100x claim requires every candidate to prove the same source predicate, at the same security target, on the same hardware, while including every conversion and boundary operation. The first matrix should contain these pipelines:

Lane	Construction	Admission rule
C-AIR	ordinary gate-by-gate arithmetic circuit/AIR	independent source conformance
D-NOWRAP	positive residual with certified partial bounds	all no-wrap obligations discharge
D-BOOLGRAPH	canonical bits and inverse-witness zero tests	bitness and source equivalence
D-HYBRID	certificate-selected residual/Boolean/shared DAG	certificate replay plus source equivalence
M-IMPL	public Modulus compiler/prover, if supplied	source, parameters, build, and raw artifacts available

The workload matrix should cross Boolean policy logic, equality-heavy batches, mixed negation and disjunction, nested finite quantifiers at several domain sizes, multi-arity signatures with public functions, shared-subterm fanout, the fixed Gabbay matrix instance, and the published swap relation. Both true and false instances are required; a prover must refuse the latter.

For every lane and size, the preserved run artifact must report:

- exact source, grounded relation, compiler/certificate hashes, field and security parameters;
- equations, multiplications, degree, witnesses, rows, columns, lookup/table cells, and every range or zero-test gadget;
- source compilation, witness generation, proving and verification time with warmup, sample count, distribution, and peak memory;

- proof bytes, public-input bytes, preprocessing material, and verifier work;
- for FHE, encryption, evaluation, conversion/bootstrapping, threshold or private zero decision, decryption, ciphertext bytes, occupancy, depth, and measured remaining noise;
- machine, operating system, compiler revisions, build flags, thread count, accelerator residency, and raw samples.

Only the complete wall-clock and resource columns can support an end-to-end cost ratio. The 30-to-13 equation result, 26-to-13 multiplication result, eight-versus-fifteen BFV ledger, and 6-versus-9 hybrid FHE ledger are hypotheses for that experiment, not substitutes for it.

The 1-3 second finality claim requires a different matrix. At minimum it must state the consensus and fault model, validator count and geography, block and proof size, data-availability path, network delay and loss model, batching and sequencing policy, hardware, confirmation rule, and latency distribution under load and faults.

Logic arithmetization can change proving cost, but finality is also a consensus, networking, execution, and data-availability property.

6.7 Constructive public comparison

As inspected on 22 July 2026, the public BitLogic repository contains its whitepaper PDF but no compiler, witness generator, prover, verifier, release, benchmark driver, parameter manifest, or raw result bundle. The project site advertises 10-100x cost reduction and 1-3 second finality. No public adapter exists that can run those claims through the matrix above. The counterexample in this report is therefore an attack on a published relation, not on unavailable private code or a deployed chain.

Axis	Reviewed public Modulus evidence	DREGG artifact in this study
Source semantics	Printed equations; the published swap product admits the proved bypass.	Kernel-checked corrected FOL, finite-signature source semantics, conformance corpus.
Compiler	No public compiler implementation found.	Lean FOL-to-IR compiler, executable Rust front end, canonical Python reference compiler.
Optimization	No public derivation or cost certificate found.	Replayable Lean rewrite certificates and independently checked hybrid search artifact.
Proof path	Architecture prose names zkEVM, STARK, and SNARK components.	Live IR-v2 descriptor, Rust prove/verify roundtrip, explicit FRI/deployment assumptions.
Independent checker	No public checker source found.	Independent SML checker, HOL4 soundness, proof-producing CakeML translation.
Encrypted logic	No public FHE logic compiler found.	Same-opening proof/FHE theorem, certified mixed plan, real BFV execution.
Performance	Headline ratios without a reviewed reproducibility bundle.	Exact symbolic savings and scoped measurements; no end-to-end winner claimed.

FORMAL ARTIFACT - the public lead DREGG's present advantage is auditable semantics and compositional evidence: source, countermodels, repair theorems, compilers, certificates, fixtures, and boundary assumptions can all be inspected. It is not yet an established advantage in prover latency, proof size, monetary cost, or chain finality.

SECTION 7

Conclusions: from a broken shortcut to a certified compiler family

The public BitLogic shortcut fails. The underlying research direction does not. The useful result of this audit is a stronger architecture than the claim that prompted it.

DECISIVE RESULT We now have a coherent path from matrix FOL to sound compiler semantics, and from general bounded finite FOL to live DREGG descriptors, certified local optimization, and matching encrypted plans. We also have a genuinely higher-order intensional route: typed shared nets form a cartesian closed category, and the structural compiler preserves the chosen CCC operations. For arbitrary small CCCs, Yoneda supplies the correct general embedding into presheaves. None of these results depends on pretending that FOL or a finite polynomial table is the primitive form of logic.

7.1 The corrected answer to the original claim

Gabbay's theorem is not false in its actual setting. Over nonnegative rationals, zero residuals have the intended meaning: conjunction and bounded universal quantification are sums; disjunction and bounded existential quantification are products. The error is to transport those equations into a prime field without transporting their semantic hypotheses.

The repaired compiler makes that transport explicit. It either proves numerator and denominator bounds before field projection, or converts truth to canonical bits with checked zero witnesses. `GabbayMatrixCompiler` reconstructs the actual matrix language and interpolation mechanism; `GabbayMatrixBridge` proves the two formal presentations coherent; and `GabbayDescriptorIR2` supplies a fixed-shape private Skolem-witness relation in the live proof grammar. `GabbayDescriptorIR2PublicBoundary` proves that this descriptor has zero public inputs and no hash or range sites: acceptance does not attest an independently named external table, and the fixed instance is not interpolation-performance evidence. `FiniteSignatureFOLDescriptorIR2` then extends the live route to many relation symbols, total public function symbols, equality, all connectives, and nested finite quantifiers.

BitLogic takes neither repaired route in its released equations. It reverses the zero-true connective interpretation and multiplies obligations that must all hold. The published swap product therefore accepts a displayed counterexample. That is not a speculative concern or a disagreement about notation: it is a wrong acceptance relation.

7.2 The novel contribution that emerged

The strongest reusable construction is **certified change of presentation**. One statement may be realized by different evidence objects:

source judgment $\rightarrow$ residual $\rightarrow$ Boolean graph $\rightarrow$ shared net $\rightarrow$ AIR or FHE plan.

Each arrow carries semantic equivalence, a fail-closed checker or proof, and an explicit resource bound.

Composition of such arrows is itself certified. This turns backend choice into proof-producing compilation rather than folklore. The deterministic optimizer realizes the idea for checked Boolean rewrites; the reference Pareto

search chooses residual or Boolean representations per grounded subformula; and the hybrid proof/FHE certificate proves both backends decide the same raw opening.

This is also where the verified performance results come from. On named formal specimens, checked factor sharing reduces equations from 30 to 13, multiplications from 26 to 13, and witnesses from 17 to 8. The mixed BFV plan reduces multiplications from 9 to 6 and depth from 4 to 2 while exposing one boundary zero decision. Intensional sharing reduces duplicated work from 16 to 8 nodes and total graph size from 19 to 13. These are real 1.5–2.3x structural improvements on the measured axes. They are not evidence for a universal 10–100x prover improvement. In particular, the first result is an abstract materialized BoolGraph cost ledger, not a reduction in the current live DescriptorIR2 prover, whose formula is a nested expression without intermediate BoolGraph columns. The BFV comparison does not implement or price its encrypted residual zero-test. Neither result is end-to-end.

Separately, the actual live finite-logic prover has an absolute capture on seven small one-row workloads: proof medians 2.22–26.83 ms, verification medians 0.28–0.80 ms, and proof sizes 9.3–13.1 KB on a contended Apple M2 Max. The nonmonotonic observations establish an executable baseline, not a speedup, scaling law, Modulus comparison, or finality claim.

7.3 Higher order and the CCC question

The CCC line is no longer only a no-go theorem.

- The finite extensional polynomial model remains exact but dense. It gives a specification semantics for finite equality-HOL, not a succinct compiler for arbitrary higher-order programs.
- The intensional construction supplies the missing operational answer. Typed STLC terms compile to shared nets; beta, eta, and product reductions carry local evidence; those nets form a genuine semantic quotient category with terminal objects, products, and exponentials; and the compiler is a strong-CCC functor for the formalized fragment.
- Every small CCC embeds fully faithfully by Yoneda into its presheaf category, with terminal objects, binary products, and representable exponentials preserved. This is the correct arbitrary-small-CCC generalization, but it is generally infinite and says nothing by itself about finite-field succinctness.
- The global category of presentations whose meanings vary cannot be CCC: it has no weak terminal object. After fixing the source meaning, the fibre of evidence families is a CCC. This is the precise fibred replacement for the impossible global claim.

Thus arbitrary CCC semantics can be embedded; arbitrary CCCs cannot in general be squeezed faithfully and succinctly into one finite polynomial universe. The remaining compiler problem is intensional: choose finite representations of the part of a presheaf, net, or evidence family that a concrete protocol must expose.

7.4 Proof objects beyond polynomials

Exact descent explains when compatible local evidence determines a global object. The new transcript theorem adds the sampled layer without asserting the false deterministic total-column bridge. A committed invalid family may pass only through an OOD exceptional event, a low-degree exceptional event, or a query vector that misses every obstruction. With point binding and modularly reduced queries, the sharp composed bound is

$$\varepsilon_{\text{OOD}} + \varepsilon_{\text{LDT}} + \left((1 - \delta) + \frac{\delta}{N} \right)^k$$

when $N \bmod m = 1$. Additional transcript coordinates product out of the query marginal, while OOD and folding events may still inspect the complete transcript. This is the right form of a topos/descent contribution to a STARK: category theory identifies the global obstruction; coding theory makes it dense; commitments bind the local

views; and the transcript theorem prices the chance that sampling misses it. A topos theorem does not replace a hash, PCS, or FRI argument, but it can tell those mechanisms exactly what they must bind and test.

7.5 Executable assurance now present

- The Rust finite-logic front end emits canonical live descriptor bytes, pins source and descriptor digests, exhaustively compares source and relation on its finite specimen, proves and verifies one honest trace through the live backend, and fails closed on malformed types, domains, encodings, and limits.
- Its captured one-row live proofs record absolute prover, verifier, and proof- size measurements; contention and nonmonotonicity are retained rather than interpreted as a performance ratio.
- The independent Python reference compiler checks versioned residual, Boolean, and hybrid artifacts and recomputes selected derivations rather than trusting their claimed cost.
- The independent Standard ML checker passed 15/15 version-1 fixtures. HOL4 proves that accepted bytes decode to equivalent source and target semantics, reports zero theory axioms, and produces CakeML translation certificates for the checker functions.
- The BFV executor runs the corresponding Boolean and bounded residual fragments on real ciphertexts. The formal hybrid certificate proves semantic agreement on one opening and exposes multiplication, depth, and boundary decision costs.

This is already a toolchain, not a white-paper sketch. Its remaining trust boundaries are also explicit: the Rust implementations are not yet proved to refine the Lean models; the CakeML certificate has not yet been turned into a shipped native checker binary; the BFV path still needs proved noise and parameter security and an implemented encrypted residual zero-test; the symbolic optimizer still needs a certified lowering that materializes its savings in the live descriptor/prover; and the robust descent theorem still needs a complete deployed FRI/PCS reduction and verifier correspondence.

7.6 What may be claimed now

COMPARATIVE RESULT DREGG surpasses the reviewed public Modulus/BitLogic material in publicly checkable semantics, construction depth, falsification coverage, executable tooling, certificate discipline, and reproducibility. The public BitLogic relation is refuted; DREGG's corrected relations are proved exact under stated premises. DREGG also demonstrates certified structural savings. It does not yet establish a 10–100x end-to-end proving advantage, 1–3 second chain finality, or runtime superiority to an unpublished Modulus implementation.

Finality is a consensus, networking, data-availability, and validator claim; it does not follow from replacing a circuit connective. Likewise, a symbolic multiplication count is not a prover benchmark. A defensible performance claim must measure the same workload through baseline AIR, repaired residual, Boolean, and certified hybrid paths, including witness generation, proof generation, verification, proof bytes, range checks, zero decisions, and every conversion boundary.

7.7 Constructive offer

TO MODULUSZK AND CULTDAO Keep the useful matrix-language insight. Replace the finite-field equations with one of the proved repairs. Conjoin independent obligations. Publish the exact source language, compiler, relation, witness generator, prover, verifier, parameters, corpus, raw measurements, and exclusions. Import the counterexamples as permanent regression tests. Then compare the repaired implementation against the certified residual, Boolean, shared, and hybrid constructions here on identical workloads.

BOUNDARY This report establishes a specification-level failure in the released BitLogic relation, not an exploit against private or deployed software. It also establishes positive formal and executable DREGG constructions, not a production security certification. Those two boundaries make the conclusion stronger: the criticism is reproducible, and the alternative is inspectable.

The research program that survives is not “logic bypasses circuits.” It is: compile judgments into the cheapest sound local presentation, attach evidence for every translation, and let proof systems, encrypted runtimes, and future backends compete inside one semantics-preserving envelope.

SECTION 8

Primary sources and inspected artifacts

1. Murdoch J. Gabbay. *Arithmetisation of computation via polynomial semantics for first-order logic*. IACR ePrint 2024/954, third revision, 12 February 2026; Journal of Applied Logics 12(6), 1481-1548, 2025. In the current revision, the principal locators used in this report are Figure 2 and Definition 2.2.8, Definition 2.3.1, Lemma 2.3.2, Theorem 2.3.4, Remark 2.3.5, and Remark 6.3.3. ePrint record and current PDF.
2. Murdoch J. Gabbay and Andrew Mendelsohn. *SNARKs for First Order Logic*. Workshop on Cryptographic Tools for Blockchains, affiliated with EUROCRYPT 2026, Rome, 9 May 2026, 46 slides. slides workshop programme.
3. Mr. O’ Modulus. *BitLogic from Modulus: zkFOL Bitcoin: An Engineering Whitepaper for Formal Logic, Zero-Knowledge, and Decentralized Finance on Bitcoin*. 21 November 2025. Public PDF and repository inspected at commit `26f125b37f0442d92991e3066095b1dfef1b0ce4`. pinned GitHub tree.
4. ModulusZK. Official homepage and documentation, inspected 22 July 2026. homepage documentation architecture glossary.
5. ModulusZK. Public posts cited in the text, 1 December 2025 and 21 July 2026. 1 December 2025 post 21 July 2026 post.
6. Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. *Fast Reed-Solomon Interactive Oracle Proofs of Proximity*. ICALP 2018, LIPIcs 107, Article 14, 1-17. DOI.
7. Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. *Scalable, transparent, and post-quantum secure computational integrity*. IACR ePrint 2018/046. ePrint.
8. Junfeng Fan and Frederik Vercauteren. *Somewhat Practical Fully Homomorphic Encryption*. IACR ePrint 2012/144. ePrint.
9. Emily Riehl. *Category Theory in Context*. Dover Publications, 2016. See Chapter 2 for representability and the Yoneda lemma. author’s book page.

10. J. Lambek and P. J. Scott. *Introduction to Higher-Order Categorical Logic*. Cambridge Studies in Advanced Mathematics 7, Cambridge University Press, 1986. publisher front matter.
11. J. M. E. Hyland, P. T. Johnstone, and A. M. Pitts. *Tripos Theory*. Mathematical Proceedings of the Cambridge Philosophical Society 88 (1980), 205-232. DOI.
12. Jaap van Oosten. *Realizability: An Introduction to its Categorical Side*. Studies in Logic and the Foundations of Mathematics 152, Elsevier, 2008. publisher page.
13. Wouter Pieter Stekelenburg. *Categories of assemblies for realizability*. arXiv:1307.0663, 2013. arXiv.
14. S. A. Terwijn. *Computability in partial combinatory algebras*. arXiv:1910.09258, third revision, 5 February 2020. arXiv.
15. Liron Cohen, Étienne Miquey, and Ross Tate. *Evidenced Frames: A Unifying Framework Broadening Realizability Models*. LICS 2021. DOI.
16. Liron Cohen, Ariel Grunfeld, Dominik Kirst, and Étienne Miquey. *Syntactic Effectful Realizability in Higher-Order Logic*. LICS 2025. DOI arXiv.
17. Liron Cohen, Ariel Grunfeld, Dominik Kirst, and Étienne Miquey. *From Partial to Monadic: Combinatory Algebra with Effects*. FSCD 2025, LIPIcs 337, Article 14. DOI artifact.
18. Liron Cohen and Tomer Samara. *Evidence-Tracked Tape Semantics for Probabilistic Computation*. FSCD 2026, LIPIcs 378, Article 13. DOI.
19. Yves Lafont. *Interaction Nets*. POPL 1990, 95-108. DOI.
20. Adrien Ragot, Thomas Seiller, and Lorenzo Tortora de Falco. *Linear Realisability over Nets: Multiplicatives*. CSL 2025, LIPIcs 326, Article 43. DOI.
21. Uriya A. First and Tali Kaufman. *Cosystolic Expansion of Sheaves on Posets with Applications to Good 2-Query Locally Testable Codes and Lifted Codes*. STOC 2024, 1749-1760. DOI arXiv, third revision.
22. Irit Dinur, Siqi Liu, and Rachel Yun Zhang. *New Codes on High Dimensional Expanders*. CCC 2025, LIPIcs 339, Article 27. DOI arXiv.
23. Hugo Delavenne, Tanguy Medevielle, and Élina Roussel. *Interactive Oracle Proofs of Proximity to Codes on Graphs*. arXiv:2501.14337, second revision, 13 May 2025. arXiv.
24. Hugo Delavenne and Louise Lallemand. *Codes on any Cayley Graph have an Interactive Oracle Proof of Proximity*. arXiv:2508.10510, 14 August 2025. arXiv.
25. Yong Kiam Tan, Magnus O. Myreen, Ramana Kumar, Anthony Fox, Scott Owens, and Michael Norrish. *The verified CakeML compiler backend*. Journal of Functional Programming 29 (2019), e2. DOI.
26. Oskar Abrahamsson, Magnus O. Myreen, Ramana Kumar, and Thomas Sewell. *Candle: A Verified Implementation of HOL Light*. ITP 2022, LIPIcs 237, Article 3. DOI project page.
27. Albert Garreta, Hendrik Waldner, Katerina Hristova, and Luca Dall'Ava. *Zinc: Succinct Arguments with Small Arithmetization Overheads from IOPs of Proximity to the Integers*. IACR ePrint 2025/316. ePrint.
28. Alireza Shirzad, Sriram Sridhar, Dimitrios Papadopoulos, and Charalampos Papamanthou. *Relaxed Modular PCS from Arbitrary PCS and Applications to SNARKs for Integers*. IACR ePrint 2026/347, revised 20 June 2026. ePrint.
29. Alexander Abdugafarov, Albert Garreta, Amit Kumar, Michał Osadnik, Psi Vesely, Ilia Vlasov, and Kai Zhe Zheng. *Zinc+: SNARKs for Polynomial Rings*. IACR ePrint 2026/855, revised 9 May 2026. ePrint.
30. Marco Stronati, Denis Firsov, Antonio Locascio, and Benjamin Livshits. *Clap: a Semantic-Preserving Optimizing eDSL for Plonkish Proof Systems*. arXiv:2405.12115, second revision, 11 July 2024. arXiv.
31. Simon Guilloud, Sankalp Gambhir, Andrea Gilot, and Viktor Kunčák. *Mechanized HOL Reasoning in Set Theory*. ITP 2024, LIPIcs 309, Article 18. DOI.
32. Raghav Malik, Vedant Paranjape, and Milind Kulkarni. *Circuit Optimization using Arithmetic Table Lookups*. Proceedings of the ACM on Programming Languages 9, PLDI (2025), Article 159, 301-323. DOI.

33. Song Bian, Yintai Sun, Zian Zhao, Haowen Pan, Mingzhe Zhang, and Zhenyu Guan. *Libra: Pattern-Scheduling Co-Optimization for Cross-Scheme FHE Code Generation over GPGPU*. USENIX Security 2026, prepublication page inspected July 2026. USENIX.
34. Zhihao Li, Hongyu Wang, Yuan Zhao, Lichun Li, Zhiwei Wang, Jiaxing He, Changzheng Wei, Ying Yan, and Lifeng Guo. *BatchBoot: Fast Batched Bootstrapping for TFHE scheme and Practical Applications*. USENIX Security 2026, prepublication page inspected July 2026. USENIX.
35. Siddharth Bhat, Alex Keizer, Chris Hughes, Andrés Goens, and Tobias Grosser. *Verifying Peephole Rewriting in SSA Compiler IRs*. ITP 2024, LIPIcs 309, Article 9. DOI artifact.
36. DREGG formalization sources. Integration root:
`metatheory/Dregg2/Metatheory/DirectLogicArithmetization.lean`. The report cites individual theorem names from its import closure.